

parent 指针指向 kobject 的父对象。这样一来, kobject 就会在内核中构造一个对象层次结构, 并且可以将多个对象间的关系表现出来。就如你所看到的, 这便是 sysfs 的真正面目: 一个用户空间的文件系统, 用来表示内核中 kobject 对象的层次结构。

sd 指针指向 sysfs_dirent 结构体, 该结构体在 sysfs 中表示的就是这个 kobject。从 sysfs 文件系统内部看, 这个结构体是表示 kobject 的一个 inode 结构体。

kref 提供引用计数。ktype 和 kset 结构体对 kobject 对象进行描述和分类。在下面的内容中将详细介绍它们。

kobject 通常是嵌入其他结构中的, 其单独意义其实并不大。相反, 那些更为重要的结构体, 比如定义于 <linux/cdev.h> 中的 struct cdev 中才真正需要用到 kobj 结构。

```
/* cdev structure - 该对象代表一个字符设备 */
struct cdev {
    struct kobject          kobj;
    struct module          *owner;
    const struct file_operations *ops;
    struct list_head       list;
    dev_t                  dev;
    unsigned int           count;
};
```

当 kobject 被嵌入到其他结构中时, 该结构便拥有了 kobject 提供的标准功能。更重要的一点是, 嵌入 kobject 的结构体可以成为对象层次架构中的一部分。比如 cdev 结构体就可通过其父指针 cdev->kobj.parent 和链表 cdev->kobj.entry 插入到对象层次结构中。

17.3.2 ktype

kobject 对象被关联到一种特殊的类型, 即 ktype (kernel object type 的缩写)。ktype 由 kobj_type 结构体表示, 定义于头文件 <linux/kobject.h> 中:

```
struct kobj_type {
    void (*release)(struct kobject *);
    const struct sysfs_ops *sysfs_ops;
    struct attribute **default_attrs;
};
```

ktype 的存在是为了描述一族 kobject 所具有的普遍特性。如此一来, 不再需要每个 kobject 都分别定义自己的特性, 而是将这些普遍的特性在 ktype 结构中一次定义, 然后所有“同类”的 kobject 都能共享一样的特性。

release 指针指向在 kobject 引用计数减至零时要被调用的析构函数。该函数负责释放所有 kobject 使用的内存和其他相关清理工作。

sysfs_ops 变量指向 sysfs_ops 结构体。该结构体描述了 sysfs 文件读写时的特性。有关其细节参见 17.3.9 节。

最后, default_attrs 指向一个 attribute 结构体数组。这些结构体定义了该 kobject 相关的默认属性。属性描述了给定对象的特征, 如果该 kobject 导出到 sysfs 中, 那么这些属性都将相应地作为文件而导出。数组中的最后一项必须为 NULL。

17.3.3 kset

kset 是 kobject 对象的集合体。把它看成是一个容器，可将所有相关的 kobject 对象，比如“全部的块设备”置于同一位置。听起来 kset 与 ktype 非常类似，好像没有多少实质内容。那么“为什么会需要这两个类似的东西呢？”kset 可把 kobject 集中到一个集合中，而 ktype 描述相关类型 kobject 所共有的特性，它们之间的重要区别在于：具有相同 ktype 的 kobject 可以被分组到不同的 kset。就是说，在 Linux 内核中，只有少数一些的 ktype，却有多多个 kset。

kobject 的 kset 指针指向相应的 kset 集合。kset 集合由 kset 结构体表示，定义于头文件 `<linux/kobject.h>` 中：

```
struct kset {
    struct list_head    list;
    spinlock_t          list_lock;
    struct kobject       kobj;
    struct kset_uevent_ops *uevent_ops;
};
```

在这个结构中，其中 list 连接该集合（kset）中所有的 kobject 对象，list_lock 是保护这个链表中元素的自旋锁（关于自旋锁的讨论，详见第 10 章），kobj 指向的 kobject 对象代表了该集合的基类。uevent_ops 指向一个结构体——用于处理集合中 kobject 对象的热插拔操作。uevent 就是用户事件（user event）的缩写，提供了与用户空间热插拔信息进行通信的机制。

17.3.4 kobject、ktype 和 kset 的相互关系

上文反复讨论的这一组结构体很容易令人混淆，这可不是因为它们数量繁多（其实只有三个），也不是它们太复杂（它们都相当简单），而是由于它们内部相互交织。要了解 kobject，很难只讨论其中一个结构而不涉及其他相关结构。然而在这些结构的相互作用下，会更有助你深刻理解它们之间的关系。

这里最重要的家伙是 kobject，它由 struct kobject 表示。kobject 为我们引入了诸如引用计数（reference counting）、父子关系和对象名称等基本对象道具，并且是以一个统一的方式提供这些功能。不过 kobject 本身意义并不大，通常情况下它需要被嵌入到其他数据结构中，让那些包含它的结构具有了 kobject 的特性。

kobject 与一个特别的 ktype 对象关联，ktype 由 struct kobj_type 结构体表示，在 kobject 中 ktype 字段指向该对象。ktype 定义了一些 kobject 相关的默认特性：析构行为（反构造功能）、sysfs 行为（sysfs 的操作表）以及别的一些默认属性。

kobject 又归入了称作 kset 的集合，kset 集合由 struct kset 结构体表示。kset 提供了两个功能。第一，其中嵌入的 kobject 作为 kobject 组的基类。第二，kset 将相关的 kobject 集合在一起。在 sysfs 中，这些相关的 kobject 将以独立的目录出现在文件系统中。这些相关的目录，也许是给定目录的所有子目录，它们可能处于同一个 kset。

图 17-1 描述了这些数据结构的内在关系。

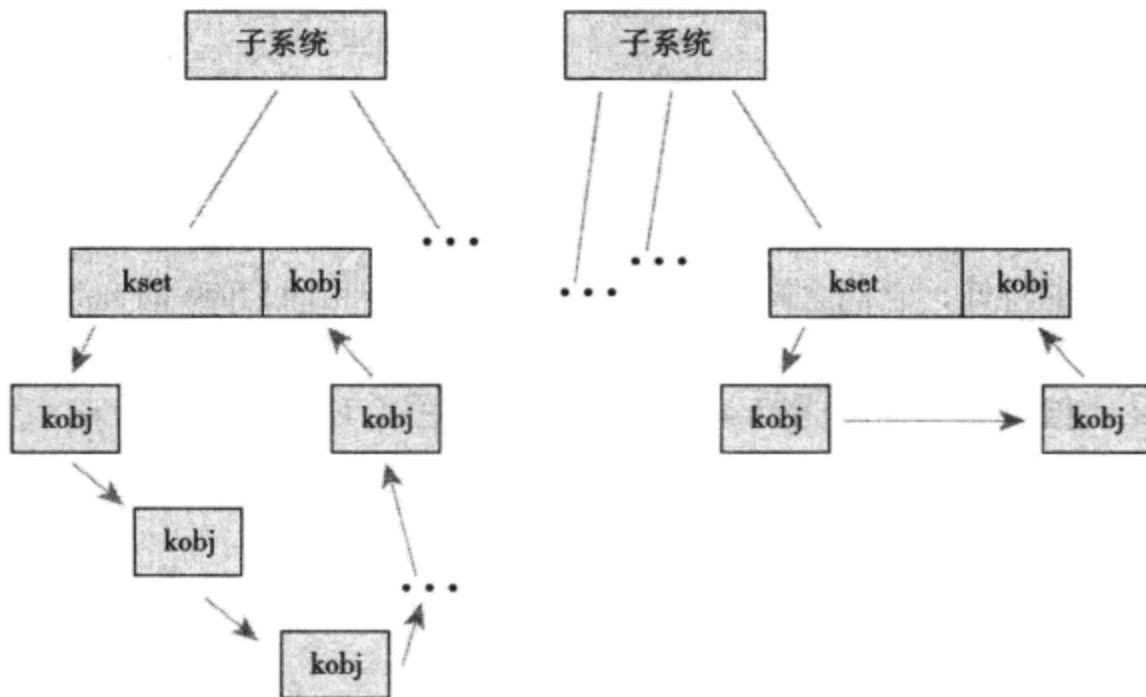


图 17-1 kobject、kset 和子系统的内在关系

17.3.5 管理和操作 kobject

当了解了 kobject 的内部基本细节后，我们来看管理和操作它的外部接口了。多数时候，驱动程序开发者并不必直接处理 kobject，因为 kobject 是被嵌入到一些特殊类型结构体中的（就如在字符设备结构体中看到的情形），而且会由相关的设备驱动程序在“幕后”管理。即便如此，kobject 并不是有意在隐藏自己，它可以出现在设备驱动代码中，或者可以在设备驱动子系统本身中使用它。

使用 kobject 的第一步需要先来声明和初始化。kobject 通过函数 kobject_init 进行初始化，该函数定义在文件 <linux/kobject.h> 中：

```
void kobject_init(struct kobject *kobj, struct kobj_type *ktype);
```

该函数的第一个参数就是需要初始化的 kobject 对象，在调用初始化函数前，kobject 必须清空。这个工作往往会在 kobject 所在的上层结构体初始化时完成。如果 kobject 未被清空，那么只需要调用 memset() 即可：

```
memset(kobj, 0, sizeof (*kobj));
```

在清零后，就可以安全的初始化 parent 和 kset 字段。例如，

```
struct kobject *kobj;

kobj = kmalloc(sizeof (*kobj), GFP_KERNEL);
if (!kobj)
    return -ENOMEM;
memset(kobj, 0, sizeof (*kobj));
kobj->kset = my_kset;
kobject_init(kobj, my_ktype);
```

这多步操作也可以由 kobject_create() 来自动处理，它返回一个新分配的 kobject：

```
struct kobject *kobject_create(void);
```

使用相当简单：

```
struct kobject *kobj;

kobj = kobject_create();
if (!kobj)
    return -ENOMEM;
```

大多数情况下，应该调用 `kobject_create()` 创建 `kobject`，或者是调用相关的辅助函数，而不是直接操作这个结构体。

17.3.6 引用计数

`kobject` 的主要功能之一就是为我们提供了一个统一的引用计数系统。初始化后，`kobject` 的引用计数设置为 1。只要引用计数不为零，那么该对象就会继续保留在内存中，也可以说是被“钉住”了。任何包含对象引用的代码首先要增加该对象的引用计数，当代码结束后则减少它的引用计数。增加引用计数称为获得（getting）对象的引用，减少引用计数称为释放（putting）对象的引用。当引用计数跌到零时，对象便可以被撤销，同时相关内存也都被释放。

1. 递增和递减引用计数

增加一个引用计数可通过 `kobject_get()` 函数完成：

```
struct kobject * kobject_get(struct kobject *kobj);
```

该函数正常情况下将返回一个指向 `kobject` 的指针，如果失败则返回 `NULL` 指针；

减少引用计数通过 `kobject_put()` 完成，这个指令也声明在 `<linux/kobject.h>` 中：

```
void kobject_put(struct kobject *kobj);
```

如果对应的 `kobject` 的引用计数减少到零，则与该 `kobject` 关联的 `ktype` 中的析构函数将被调用。

2. kref

我们深入到引用计数系统的内部去看，会发现 `kobject` 的引用计数是通过 `kref` 结构体实现的，该结构体定义在头文件 `<linux/kref.h>` 中：

```
struct kref {
    atomic_t refcount;
};
```

其中唯一的字段是用来存放引用计数的原子变量。那为什么采用结构体？这是为了便于进行类型检测。在使用 `kref` 前，你必须先通过 `kref_init()` 函数来初始化它：

```
void kref_init(struct kref *kref)
{
    atomic_set(&kref->refcount, 1);
}
```

正如你所看到的，这个函数简单地将原子变量置 1，所以 `kref` 一旦被初始化，它表示的引用计数便固定为 1。这点和 `kobject` 中的计数行为一致。

要获得对 kref 的引用，需要调用 kref_get() 函数，这个函数声明在 <linux/kref.h> 中：

```
void kref_get(struct kref *kref)
{
    WARN_ON(!atomic_read(&kref->refcount));
    atomic_inc(&kref->refcount);
}
```

该函数增加引用计数值，它没有返回值。减少对 kref 的引用，调用声明在 <Linux/kref.h> 中的函数 kref_put()：

```
int kref_put(struct kref *kref, void (*release)(struct kref *kref))
{
    WARN_ON(release == NULL);
    WARN_ON(release == (void (*)(struct kref *))kfree);

    if (atomic_dec_and_test(&kref->refcount)) {
        release(kref);
        return 1;
    }
    return 0;
}
```

该函数将使得引用计数减 1，如果计数减少到零，则要调用作为参数提供的 release() 函数。注意 WARN_ON() 声明，提供的 release() 函数不能简单地采用 kfree()，它必须是一个仅接收一个 kref 结构体作为参数的特有函数，而且还没有返回值。kref_put() 函数返回 0，但有一种情况下它返回 1，那就是在对该对象的最后一个引用减 1 时。通常情况下，kref_put() 的调用者不关心这个返回值。

开发者现在不必在内核代码中利用 atomic_t 类型来实现自己的引用计数和简单的“get”、“put”这些封装函数。对开发者而言，在内核代码中最好的方法是利用 kref 类型和它相应的辅助函数，为自己提供一个通用的、正确的引用计数机制。

上述的所有函数定义与声明分别在文件 lib/kref.c 和文件 <linux/kref.h> 中。

17.4 sysfs

sysfs 文件系统是一个处于内存中的虚拟文件系统，它为我们提供了 kobject 对象层次结构的视图。帮助用户能以一个简单文件系统的方式来观察系统中各种设备的拓扑结构。借助属性对象，kobject 可以用导出文件的方式，将内核变量提供给用户读取或写入（可选）。

虽然设备模型的初衷是为了方便电源管理而提出的一种设备拓扑结构，但是 sysfs 是颇为意外的收获。为了方便调试，设备模型的开发者决定将设备结构树导出为一个文件系统。这个举措很快被证明是非常明智的，首先 sysfs 代替了先前处于 /proc 下的设备相关文件；另外它为系统对象提供了一个很有效的视图。实际上，sysfs 起初被称为 driverfs，它早于 kobject 出现。最终 sysfs 使得我们认识到一个全新的对象模型非常有利于系统，于是 kobject 应运而生。今天所有 2.6 内核的系统都拥有 sysfs 文件系统，而且几乎都毫无例外的将其挂载在 sys 目录下。

sysfs 的诀窍是把 kobject 对象与目录项 (directory entries) 紧密联系起来，这点是通过 kobject 对象中的 dentry 字段实现的。回忆第 12 章，dentry 结构体表示目录项，通过连接 kobject 到指定的目录项上，无疑方便地将 kobject 映射到该目录上。从此，把 kobject 导出形成文件系统

就变得如同在内存中构建目录项一样简单。好了，kobject 其实已经形成了一棵树——就是我们心爱的对象模型体系。由于 kobject 被映射到目录项，同时对象层次结构也已经在内存中形成了一棵树，因此 sysfs 的生成便水到渠成般地简单了。

```

-- block
|  -- loop0 -> ../devices/virtual/block/loop0
|  -- md0 -> ../devices/virtual/block/md0
|  -- nbd0 -> ../devices/virtual/block/nbd0
|  -- ram0 -> ../devices/virtual/block/ram0
|  -- xvda -> ../devices/vbd-51712/block/xvda
-- bus
|  -- platform
|  -- serio
-- class
|  -- bdi
|  -- block
|  -- input
|  -- mem
|  -- misc
|  -- net
|  -- ppp
|  -- rtc
|  -- tty
|  -- vc
|  -- vtconsole
-- dev
|  -- block
|  -- char
-- devices
|  -- console-0
|  -- platform
|  -- system
|  -- vbd-51712
|  -- vbd-51728
|  -- vif-0
|  -- virtual
-- firmware
-- fs
|  -- ecryptfs
|  -- ext4
|  -- fuse
|  -- gfs2
-- kernel
|  -- config
|  -- dlm
|  -- mm
|  -- notes
|  -- uevent_helper
|  -- uevent_seqnum
|  -- uids
-- module
|  -- ext4
|  -- i8042
|  -- kernel
|  -- keyboard
|  -- mousedev
|  -- nbd
|  -- printk
|  -- psmouse
|  -- sch_htb
|  -- tcp_cubic
|  -- vt
|  -- xt_recent

```

图 17-2 挂载于 /sys 目录下的 sysfs 文件系统的局部视图

sysfs 的根目录下包含了至少十个目录：block、bus、class、dev、devices、firmware、fs、kernel、module 和 power。block 目录下的每个子目录都对应着系统中的一个已注册的块设备。反过来，每个目录下又都包含了该块设备的所有分区。bus 目录提供了一个系统总线视图。class 目录包含了以高层功能逻辑组织起来的系统设备视图。dev 目录是已注册设备节点的视图。devices 目录是系统中设备拓扑结构视图，它直接映射出了内核中设备结构体的组织层次。firmware 目录包含了一些诸如 ACPI、EDD、EFI 等低层子系统的特殊树。fs 目录是已注册文件系统的视图。kernel 目录包含内核配置项和状态信息，module 目录则包含系统已加载模块的信息。power 目录包含系统范围的电源管理数据。并不是所有的系统都包含所有这些目录，还有些系统含有其他目录，但在这里尚未提到。

其中最重要的目录是 devices，该目录将设备模型导出到用户空间。目录结构就是系统中实际的设备拓扑。其他目录中的很多数据都是将 devices 目录下的数据加以转换加工而得。比如，/sys/class/net/ 目录是以注册网络接口这一高层概念来组织设备关系的，在这个目录中可能会有目录 eth0，它里面包含的 devices 文件其实就是一个指回到 devices 下实际设备目录的符号连接。

随便看看你可访问到的任何 Linux 系统的 sys 目录，这种系统设备视图相当准确和漂亮，而且可以看到 class 中的高层概念与 devices 中的低层物理设备，以及 bus 中的实际驱动程序之间互相联络是非常广泛的。当你认识到这种数据是开放的，换句话说，这是内核中维持系统的很好表示方式^①时，整个经历都弥足珍贵。

17.4.1 sysfs 中添加和删除 kobject

仅仅初始化 kobject 是不能自动将其导出到 sysfs 中的，想要把 kobject 导入 sysfs，你需要用到函数 kobject_add()：

```
int kobject_add(struct kobject *kobj, struct kobject *parent, const char *fmt, ...);
```

kobject 在 sysfs 中的位置取决于 kobject 在对象层次结构中的位置。如果 kobject 的父指针被设置，那么在 sysfs 中 kobject 将被映射为其父目录下的子目录；如果 parent 没有设置，那么 kobject 将被映射为 kset->kobj 中的子目录。如果给定的 kobject 中 parent 或 kset 字段都没有被设置，那么就认为 kobject 没有父对象，所以就会被映射成 sysfs 下的根级目录。这往往不是你所需要的，所以在调用 kobject_add() 前 parent 或 kset 字段应该进行适当的设置。不管怎么样，sysfs 中代表 kobject 的目录名字是由 fmt 指定的，它也接受 printf() 样式的格式化字符串。

辅助函数 kobject_create_and_add() 把 kobject_create() 和 kobject_add() 所做的工作放在一个函数中：

```
struct kobject *kobject_create_and_add(const char *name, struct kobject *parent);
```

注意 kobject_create_and_add() 函数接受直接的指针 name 作为 kobject 所对应的目录名称，而 kobject_add() 使用 printf() 风格的格式化字符串。

① 如果你对 sysfs 感兴趣，你可能也会对 HAL 感兴趣，它是一个硬件抽象层，可以在 <http://hal.freedesktop.org/wiki/software/hal> 找到它。HAL 基于 sysfs 中的数据建立起了一个内存数据库，将 class 概念、设备概念和驱动概念联系在一起。在这些数据之上，HAL 提供了丰富的 API 以使得应用程序更灵活。

从 sysfs 中删除一个 kobject 对应文件目录，需使用函数 `kobject_del()`：

```
void kobject_del(struct kobject *kobj);
```

上述这些函数都定义于文件 `lib/kobject.c` 中，声明于头文件 `<linux/kobject.h>` 中。

17.4.2 向 sysfs 中添加文件

我们已经看到 kobject 被映射为文件目录，而且所有的对象层次结构都优雅地、一个不少地映射成 sys 下的目录结构。但是里面的文件是什么？sysfs 仅仅是一个漂亮的树，但是没有提供实际数据的文件。

1. 默认属性

默认的文件集合是通过 kobject 和 kset 中的 `ktype` 字段提供的。因此所有具有相同类型的 kobject 在它们对应的 sysfs 目录下都拥有相同的默认文件集合。`kobj_type` 字段含有一个字段——`default_attrs`，它是一个 attribute 结构体数组。这些属性负责将内核数据映射成 sysfs 中的文件。

attribute 结构体定义在文件 `<linux/sysfs.h>` 中：

```
/* attribute 结构体 - 内核数据映射成 sysfs 中的文件 */
struct attribute {
    const char      *name;      /* 属性名称 */
    struct module    *owner;     /* 所属模块，如果存在 */
    mode_t           mode;      /* 权限 */
};
```

其中名称字段提供了该属性的名称，最终出现在 sysfs 中的文件名就是它。`owner` 字段在存在所属模块的情况下指向其所属的 module 结构体。如果一个模块没有该属性，那么该字段为 NULL。`mode` 字段类型为 `mode_t`，它表示了 sysfs 中该文件的权限。对于只读属性而言，如果是所有人都可读它，那么该字段被设为 `S_IRUGO`；如果只限于所有者可读，则该字段被设置为 `S_IRUSR`。同样对于可写属性，可能会设置该字段为 `S_IRUGO | S_IWUSR`。sysfs 中的所有文件和目录的 uid 与 gid 标志均为零。

虽然 `default_attrs` 列出了默认的属性，`sysfs_ops` 字段则描述了如何使用它们。`sysfs_ops` 字段指向了一个定义于文件 `<linux/sysfs.h>` 的同名的结构体：

```
struct sysfs_ops {
    /* 在读 sysfs 文件时该方法被调用 */
    ssize_t (*show) (struct kobject *kobj,
                    struct attribute *attr,
                    char *buffer);

    /* 在写 sysfs 文件时该方法被调用 */
    ssize_t (*store) (struct kobject *kobj,
                    struct attribute *attr,
                    const char *buffer,
                    size_t size);
};
```


当从用户空间读取 sysfs 的项时调用 show() 方法。它会拷贝由 attr 提供的属性值到 buffer 指定的缓冲区中，缓冲区大小为 PAGE_SIZE 字节；在 x86 体系中，PAGE_SIZE 为 4096 字节。该函数如果执行成功，则将返回实际写入 buffer 的字节数；如果失败，则返回负的错误码。

store() 方法在写操作时调用，它会从 buffer 中读取 size 大小的字节，并将其存放入 attr 表示的属性结构体变量中。缓冲区的大小总是为 PAGE_SIZE 或更小些。该函数如果执行成功，则将返回实际从 buffer 中读取的字节数；如果失败，则返回负数的错误码。

由于这组函数必须对所有的属性都进行文件 I/O 请求处理，所以它们通常需要维护某些通用映射来调用每个属性所特有的处理函数。

2. 创建新属性

通常来讲，由 kobject 相关 ktype 所提供的默认属性是充足的，事实上，因为所有具有相同 ktype 的 kobject，在本质上区别不大的情况下，都应是相互接近的。也就是说，比如对于所有的分区而言，它们完全可以具有同样的属性集合。这不但可以让事情简单，有助于代码合并，还使类似对象在 sysfs 目录中外观一致。

但是，有时在一些特别情况下会碰到特殊的 kobject 实例。它希望（甚至是必须）有自己的属性——也许是通用属性没包含那些需要的数据或者函数。为此，内核为能在默认集合之上，再添加新属性而提供了 sysfs_create_file() 接口：

```
int sysfs_create_file(struct kobject *kobj, const struct attribute *attr);
```

这个接口通过 attr 参数指向相应的 attribute 结构体，而参数 kobj 则指定了属性所在的 kobject 对象。在该函数被调用前，给定的属性将被赋值，如果成功，该函数返回零，否则返回负的错误码。

注意，kobject 中 ktype 所对应的 sysfs_ops 操作将负责处理新属性。现有的 show() 和 store() 方法必须能够处理新属性。

除了添加文件外，还有可能需要创建符号连接。在 sysfs 中创建一个符号连接相当简单：

```
int sysfs_create_link(struct kobject *kobj, struct kobject *target, char *name);
```

该函数创建的符号连接名由 name 指定，连接则由 kobj 对应的目录映射到 target 指定的目录。如果成功该函数返回零，如果失败返回负的错误码。

3. 删除新属性

删除一个属性需通过函数 sysfs_remove_file() 完成：

```
void sysfs_remove_file(struct kobject *kobj, const struct attribute *attr);
```

一旦调用返回，给定的属性将不再存在于给定的 kobject 目录中。另外由 sysfs_create_link() 创建的符号连接可通过函数 sysfs_remove_link() 删除：

```
void sysfs_remove_link(struct kobject *kobj, char *name);
```

调用一旦返回，在 kobj 对应目录中的名为 name 的符号连接将不复存在。

上述的四个函数在文件 <linux/kobject.h> 中声明；sysfs_create_file() 和 sysfs_remove_file()

函数定义于文件 `fs/sysfs/file.c` 中；`sysfs_create_link()` 和 `sysfs_remove_link()` 函数定义于文件 `fs/sysfs/symlink.c` 中。

4. sysfs 约定

当前 `sysfs` 文件系统代替了以前需要由 `ioctl()`（作用于设备节点）和 `procfs` 文件系统完成的功能。目前，在合适目录下实现 `sysfs` 属性这样的功能的确别具一格。比如利用在设备映射的 `sysfs` 目录中添加一个 `sysfs` 属性，代替在设备节点上实现一新的 `ioctl()`。采用这种方法避免了在调用 `ioctl()` 时使用类型不正确的参数和弄乱 `/proc` 目录结构。

但是为了保持 `sysfs` 干净和直观，开发者必须遵从以下约定。

首先，`sysfs` 属性应该保证每个文件只导出一个值，该值应该是文本形式而且映射为简单 C 类型。其目的是为了避免数据的过度结构化或太凌乱，现在 `/proc` 中就混乱而不具有可读性。每个文件提供一个值，这使得从命令行读写变得简洁，同时也使 C 语言程序轻易地将内核数据从 `sysfs` 导入到自身的变量中去。但有些时候，一值一文件的规则不能很有效地表示数据，那么可以将同一类型的多个值放入一个文件中。不过这时需要合理地表述它们，比如利用一个空格也许就可使其意义清晰明了。总的来讲，应考虑 `sysfs` 属性要映射到独立的内核变量（正如通常所做），而且要记住应保证从用户空间操作简单，尤其是从 `shell` 操作简单。

其次，在 `sysfs` 中要以一个清晰的层次组织数据。父子关系要正确才能将 `kobject` 层次结构直观地映射到 `sysfs` 树中。另外，`kobject` 相关属性同样需要正确，并且要记住 `kobject` 层次结构不仅仅存在于内核，而且也要作为一个树导出到用户空间，所以要保证 `sysfs` 树健全无误。

最后，记住 `sysfs` 提供内核到用户空间的服务，这多少有些用户空间的 ABI（应用程序二进制接口）的作用。用户程序可以检测和获得其存在性、位置、取值以及 `sysfs` 目录和文件的行为。任何情况下都不应改变现有的文件，另外更改给定属性，但保留其名称和位置不变无疑是在自找麻烦。

这些简单的约定保证 `sysfs` 可为用户空间提供丰富和直观的接口。正确使用 `sysfs`，其他应用程序的开发者绝不会对你的代码抱有微辞，相反会赞美它。

17.4.3 内核事件层

内核事件层实现了内核到用户的消息通知系统——就是建立在上文一直讨论的 `kobject` 基础之上。在 2.6.0 版本以后，显而易见，系统确实需要一种机制来帮助将事件传出内核输送到用户空间，特别是对桌面系统而言，因为它需要更完整和异步的系统。为此就要让内核将其事件压到堆栈：硬盘满了！处理器过热了！分区挂载了！

早期的事件层没有采用 `kobject` 和 `sysfs`，它们如过眼烟云，没有存在多久。现在的事件层借助 `kobject` 和 `sysfs` 实现已证明相当理想。内核事件层把事件模拟为信号——从明确的 `kobject` 对象发出，所以每个事件源都是一个 `sysfs` 路径。如果请求的事件与你的第一个硬盘相关，那么 `/sys/block/hda` 便是源树。实质上，在内核中我们认为事件都是从幕后的 `kobject` 对象产生的。

每个事件都被赋予了一个动词或动作字符串表示信号。该字符串会以“被修改过”或“未挂载”等词语来描述事件。

最后，每个事件都有一个可选的负载（payload）。相比传递任意一个表示负载的字符串到用户空间而言，内核事件层使用 sysfs 属性代表负载。

从内部实现来讲，内核事件由内核空间传递到用户空间需要经过 netlink。netlink 是一个用于传送网络信息的多点传送套接字。使用 netlink 意味着从用户空间获取内核事件就如同在套接字上堵塞一样易如反掌。方法就是用户空间实现一个系统后台服务用于监听套接字，处理任何读到的信息，并将事件传送到系统栈里。对于这种用户后台服务来说，一个潜在的目的就是将事件融入 D-BUS 系统^①。D-BUS 系统已经实现了一套系统范围的消息总线，这种总线可帮助内核如同系统中其他组件一样地发出信号。

在内核代码中向用户空间发送信号使用函数 kobject_uevent()：

```
int kobject_uevent(struct kobject *kobj, enum kobject_action action);
```

第一个参数指定发送该信号的 kobject 对象。实际的内核事件将包含该 kobject 映射到 sysfs 的路径。

第二个参数指定了描述该信号的“动作”或“动词”。实际的内核事件将包含一个映射成枚举类型 kobject_action 的字符串。该函数不是直接提供一个字符串，而是利用一个枚举变量来提高可重用性和保证类型安全，而且也消除了打字错误或其他错误。该枚举变量定义于文件 <linux/kobject_uevent.c> 中，其形式为 KOBJ_foo。当前值包含 KOBJ_MOUNT、KOBJ_UNMOUNT、KOBJ_ADD、KOBJ_REMOVE 和 KOBJ_CHANGE 等，这些值分别映射为字符串“mount”、“unmount”、“add”、“remove”和“change”等。当这些现有的值不够用时，允许添加新动作。

使用 kobject 和属性不但有利于很好的实现基于 sysfs 的事件，同时也有利于创建新 kobjects 对象和属性来表示新对象和数据——它们尚未出现在 sysfs 中。

这两个函数分别定义和声明于文件 lib/kobject_uevent.c 与文件 <linux/kobject.h> 中。

17.5 小结

本章中，我们考察的内核功能涉及设备驱动的实现和设备树的管理，包括模块、kobject（以及相关的 kset 和 ktype）和 sysfs。这些功能对于设备驱动程序的开发来说是非常重要的，因为这能够让他们写出更为模块化、更为高级的驱动程序。

本章讨论了内核中我们要学习的最后一个子系统，从下面开始要介绍一些普遍的但却重要的主题，这些主题是任何一个内核开发者都需要了解的，首先要讲的就是调试！

^① 想了解 D-BUS 更多的信息，参见 <http://dbus.freedesktop.org/>。

第 18 章

调 试

调试工作艰难是内核级开发区别于用户级开发的一个显著特点。相比于用户级开发，内核调试的难度确实要艰苦得多。更可怕的是，它带来的风险比用户级别更高，内核的一个错误往往立刻就能让系统崩溃。

驾驭内核调试的能力（当然，最终是为了能够成功地开发内核）很大程度上取决于经验和对整个操作系统的把握。没错，玉树临风可能会对别的事情有帮助，但是调试内核的关键还是在于你对内核的深刻理解。然而我们必须找到可以开始着手的地方，所以，在这一章里我们从调试内核的一种可能步骤开始。

18.1 准备开始

内核调试往往是一个令人挠头不已的漫长过程。不少 bug 已经让整个开发社区几个月都食不甘味了。幸运的是，在这些费劲的问题中也有不少比较简单，而且容易消灭的小 bug。运气好时，你可能面对的是些简单的小 bug。开始做一些调查之前，不会清楚到底面对的是什么。现在，需要的只是：

- 一个 bug。听起来很可笑，但确实需要一个确定的 bug。如果错误总是能够重现的话，那对我们会有很大的帮助（有一部分错误确实如此）。然而不幸的是，大部分 bug 通常都不是行为可靠而且定义明确的。
- 一个藏匿 bug 的内核版本。如果你知道这个 bug 最早出现在哪个内核版本中那就再理想不过了。如果你还不知道的话，别着急，本章会教你一个快速找出这个 bug 首先出现在哪个内核版本中的方法。
- 相关内核代码的知识和运气。调试内核其实是一个棘手的问题。不过对周围的代码理解得越多，调试起来也就越轻松。

本章中的大多数方法都假定能够让 bug 重现。因此，想要成功地进行调试，就取决于是否能让这些错误重现。如果不能，消灭 bug 就只能通过抽象出问题，再从代码中搜索蛛丝马迹来进行。虽然有时也得这么做，但如果你能够让错误重现，成功的机会要大许多。

有一些 bug 存在而且有人没办法让它重现，这听起来可能感觉挺奇怪。在用户级的程序里，bug 常常表现得很直截了当。比如，执行 foo 就会让程序立即产生核心信息转储（dump core）。但是内核中的 bug 表现却不是那么清晰。内核、用户程序和硬件之间的交互常常会很微妙。一个竞争条件可能在几百万次的算法迭代中才露出一次狰狞的面孔。设计不佳的（甚至是包含错误的）代码在某些系统上可能还让人可以忍受，而在其他的一些系统中却表现得相当糟糕。在一些特定的配置、一些特定的机器上，通常都需要付出额外的努力来触发某个 bug，不然的话，根本

看不到它。在跟踪 bug 的时候，掌握的信息越多越好。许多时候，当可以精确地重现一个 bug 的时候，就已经成功了一大半了。

18.2 内核中的 bug

内核中的 bug 多种多样。它们的产生可以有无数的原因，同时它们的表象也变化多端。从明白无误的错误代码（比如，没有把正确的值存放在恰当的位置）到同步时发生的错误（比如，共享变量锁定不当），再到错误地管理硬件（比如，给错误的控制寄存器发送错误的指令）；从降低所有程序的运行性能到毁坏数据再到使得系统处于死锁状态，都可能是 bug 发作时的症状。

从隐藏在源代码中的错误到展现在目击者面前的 bug，往往是经历一系列连锁反应的事件才可能触发的。举个例子，一个被共享的结构体，如果它没有引用计数，那么它就有可能会引发竞争条件。因为没有引用计数的话，一个进程可以在另外一个进程仍然需要使用该结构的时候就释放掉它。继而，第二个进程就有可能试图通过无效的指针去使用一个不存在的数据结构。这样做可能导致引用一个空指针，也可能导致读出一些垃圾数据，还可能并不产生什么恶果（如果该数据并没有被其他什么覆盖的话）。引用空指针会导致产生一个 oops，而垃圾数据可能会导致系统崩溃（这种情形比 oops 还坏）。用户报告了 oops 或系统的错误现象之后，开发者回过头来观察错误情形，发现在释放数据之后还会对它进行读写，存在着一个竞争条件，于是就会进行修正，给这个共享的结构加上适当的引用计数。

内核调试听起来很难，但事实上 Linux 内核与其他大型的软件项目也没有什么太大的不同。内核确实有一些独特的问题需要考虑，像定时限制和竞争条件等，它们都是允许多个线程在内核中同时运行产生的结果。

18.3 通过打印来调试

内核提供的打印函数 `printk()` 和 C 库提供的 `printf()` 函数功能几乎相同。实际上，在本书中我们都没有用到这两个函数的不同部分。从它实现的大部分意图来说，这个名字很不错，`printk()` 就是内核的格式化打印函数。但是，`printk()` 确实还有一些自身特殊的功能。

18.3.1 健壮性

健壮性是 `printk()` 函数最容易让人们接受的一个特质。任何时候，任何地方都能调用它，内核中的 `printk()` 比比皆是。可以在中断上下文和进程上下中被调用；可以在任何持有锁时被调用；可以在多处理器上同时被调用，而且调用者连锁都不必使用。

它是一个弹性极佳的函数。这一点相当重要，`printk()` 之所以这么有用，就在于它随时都能被调用。

`printk()` 函数的健壮躯壳下也难免会有漏洞。在系统启动过程中，终端还没有初始化之前，在某些地方不能使用它。不过说实在的，如果终端没有初始化，你又能输出到什么地方去呢？

这一般不是一个什么问题，除非你要调试的是启动过程最开始的那些步骤（比如说在负责执行硬件体系结构相关的初始化动作的 `setup_arch()` 函数中）。着手进行这样的调试挑战性很强——没有任何打印函数能用，确实让问题更加棘手。

不过还是有一些可以指望的（虽然不多）。核心硬件部分的黑客依靠此时能够工作的硬件设备（比如说一个串口）与外界通信。绝大部分人对此都不会感兴趣。解决的办法是提供一个 `printk()` 的变体函数——`early_printk()`，这个函数在启动过程的初期就具有在终端上打印的能力。它的功能与 `printk()` 完全相同，区别仅仅在于名字和能够更早地工作。不过，由于该函数在一些内核支持的硬件体系结构上无法实现，所以这种办法缺少可移植性。但是，如果所使用的硬件体系可以实现这个函数（大多数硬件体系都可以，包括 x86），它就是最好的指望。

除非在启动过程的初期就要在终端上输出，否则可以认为 `printk()` 在什么情况下都能工作。

18.3.2 日志等级

`printk()` 和 `printf()` 在使用上最主要的区别就是前者可以指定一个日志级别。内核根据这个级别来判断是否在终端上打印消息。内核把级别比某个特定值低的所有消息显示在终端上。

可以通过下面这种方式指定一个记录级别：

```
printk(KERN_WARNING "This is a warning!\n");
printk(KERN_DEBUG "This is a debug notice!\n");
printk("I did not specify a loglevel!\n");
```

`KERN_WARNING` 和 `KERN_DEBUG` 都是 `<linux/kernel.h>` 中的简单宏定义。它们扩展开是像 “<4>” 或 “<7>” 这样的字符串，加进 `printk()` 函数要打印的消息的开头。内核用这个指定的记录等级和当前终端的记录等级 `console_loglevel` 来决定是不是向终端上打印。表 18-1 列举了所有可供使用的记录等级。

表 18-1 可供使用的记录等级

记录等级	描 述
<code>KERN_EMERG</code>	一个紧急情况
<code>KERN_ALERT</code>	一个需要立即被注意到的错误
<code>KERN_CRIT</code>	一个临界情况
<code>KERN_ERR</code>	一个错误
<code>KERN_WARNING</code>	一个警告
<code>KERN_NOTICE</code>	一个普通的，不过也有可能需要注意的情况
<code>KERN_INFO</code>	一条非正式的消息
<code>KERN_DEBUG</code>	一条调试信息——一般是冗余信息

如果你没有特别指定一个记录等级，函数会选用默认的 `DEFAULT_MESSAGE_LOGLEVEL`，现在默认等级是 `KERN_WARNING`。由于这个默认值将来存在变化的可能性，所以还是应该给自己的消息指定一个记录等级。

内核将最重要的记录等级 `KERN_EMERG` 定为 “<0>”，将无关紧要的记录等级 “`KERN_DEBUG`” 定为 “<7>”。举例来说，当编译预处理完成之后，前例中的代码实际被编译成如下格式：

```
printk("<4> This is a warning!\n");
printk("<7> This is a debug notice!\n");
printk("<4> did not specify a loglevel!\n");
```


怎样给调用的 `printk()` 赋记录等级完全取决于自己。那些正式的、需要你保持的消息应该有合适的记录等级。但是那些当你试图解决一个问题时加得到处都是的调试信息（必须承认，我们都这么干而且也确实行得通），可以按照你的想法赋予记录等级。一种选择是保持终端的默认记录等级不变，给所有调试信息 `KERN_CRIT` 或更低的等级。相反，也可以给所有调试信息 `KERN_DEBUG` 等级，而调整终端的默认记录等级。两种方法各有利弊，自己拿主意吧。

18.3.3 记录缓冲区

内核消息都被保存在一个 `LOG_BUF_LEN` 大小的环形队列中。该缓冲区大小可以在编译时通过设置 `CONFIG_LOG_BUF_SHIFT` 进行调整。在单处理器的系统上其默认值是 16KB。换句话说，就是内核在同一时间只能保存 16KB 的内核消息。如果消息队列已经达到最大值，那么如果再有 `printk()` 调用时，新消息将覆盖队列中的老消息。这个记录缓冲区之所以称为环形，是因为它的读写都是按照环形队列方式进行操作。

使用环形队列有许多好处。由于同时读写环形缓冲区时，其同步问题很容易解决，所以即使在中断上下文中也可以方便地使用 `printk()`。此外，它使记录维护起来也更容易。如果有大量的消息同时产生，新消息只需覆盖掉旧消息即可。在某个问题引发大量消息的时候，记录只会覆盖掉它本身，而不会因为失控而消耗掉大量内存。而环形缓冲区的唯一缺点——可能会丢失消息，但是与简单性和健壮性的好处相比，这点代价是值得的。

18.3.4 syslogd 和 klogd

在标准的 Linux 系统上，用户空间的守护进程 `klogd` 从记录缓冲区中获取内核消息，再通过 `syslogd` 守护进程将它们保存在系统日志文件中。`klogd` 程序既可以从 `/proc/kmsg` 文件中，也可以通过 `syslog()` 系统调用读取这些消息。默认情况下，它选择读取 `/proc` 方式实现。不管是哪种方法，`klogd` 都会阻塞，直到有新的内核消息可供读出。在被唤醒之后，它会读取出新的内核消息并进行处理。默认情况下，它就是把消息传给 `syslogd` 守护进程。

`syslogd` 守护进程把它接收到的所有消息添加进一个文件中，该文件默认是 `/var/log/messages`。也可以通过 `/etc/syslog.conf` 配置文件重新指定。

在启动 `klogd` 的时候，可以通过指定 `-c` 标志来改变终端的记录等级。

18.3.5 从 printf() 到 printk() 的转换

当刚开始开发内核代码的时候，往往会把 `printk()` 输入成 `printf()`。这很正常，你无法抗拒多年来在用户级程序中使用 `printf()` 的习惯。幸而这种错误不会持续很长时间，反复出现的链接错误很快就会让你在心烦意乱中开始培养新的习惯。

在编写用户级程序的时候，你输入 `printf()` 的时候不小心输入了 `printk()`。恭喜你，成为一个真正的内核黑客的时刻终于到来了。

18.4 oops

`oops` 是内核告知用户有不幸发生的最常用的方式。由于内核是整个系统的管理者，所以它

不能采取像用户空间出现运行错误时使用的那些简单手段，因为它很难自行修复，它也不能将自己杀死。内核只能发布 oops。这个过程包括向终端上输出错误消息，输出寄存器中保存的信息并输出可供跟踪的回溯线索。内核中出现的故障很难处理，所以内核往往要经历严峻的考验才能发送出 oops 和靠它自己完成的一些清理工作。通常，发送完 oops 之后，内核会处于一种不稳定状态。举例来说，oops 发生的时候内核可能正在处理非常重要的数据。它可能持有一把锁或正在和硬件设备交互。内核必须适当地从当前的上下文环境中退出并尝试恢复对系统的控制。多数时候，这种尝试都会失败。因为如果 oops 在中断上下文时发生，内核根本无法继续，它会陷入混乱。混乱的结果就是系统死机。如果 oops 在 idle 进程（pid 为 0）或 init 进程（pid 为 1）时发生，结果同样是系统陷入混乱，因为内核缺了这两个重要的进程根本就无法工作。不过，要是 oops 在其他进程运行时发生，内核就会杀死该进程并尝试着继续执行。

oops 的产生有很多可能原因，其中包括内存访问越界或者非法的指令等。作为一个内核开发者，你将会经常处理（毫无疑问，也将导致）oops。

紧接着的是一个 oops 的实例，它是在一台 PPC 机器上的 tulip 网卡的定时器处理函数运行时发生的：

```
Oops: Exception in kernel mode, sig: 4
Unable to handle kernel NULL pointer dereference at virtual address 00000001

NIP: C013A7F0 LR: C013A7F0 SP: C0685E00 REGS: c0905d10 TRAP: 0700
Not tainted
MSR: 00089037 EE: 1 PR: 0 FP: 0 ME: 1 IR/DR: 11
TASK = c0712530[0] 'swapper' Last syscall: 120
GPR00: C013A7C0 C0295E00 C0231530 0000002F 00000001 C0380CB8 C0291B80 C02D0000
GPR08: 000012A0 00000000 00000000 C0292AA0 4020A088 00000000 00000000 00000000
GPR16: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
GPR24: 00000000 00000005 00000000 00001032 C3F7C000 00000032 FFFFFFFF C3F7C1C0
Call trace:
[c013ab30] tulip_timer+0x128/0x1c4
[c0020744] run_timer_softirq+0x10c/0x164
[c001b864] do_softirq+0x88/0x104
[c0007e80] timer_interrupt+0x284/0x298
[c00033c4] ret_from_except+0x0/0x34
[c0007b84] default_idle+0x20/0x60
[c0007bf8] cpu_idle+0x34/0x38
[c0003ae8] rest_init+0x24/0x34
```

使用 PC 的读者可能对这么多的寄存器感到惊奇（居然有 32 个之多）。你可能对 x86-32 系统更熟悉一些，在这种系统上，oops 会简单一点。但是，oops 中包含的重要信息对于所有体系结构都是完全相同的：寄存器上下文和回溯线索。

回溯线索显示了导致错误发生的函数调用链。这样我们就可以观察究竟发生了什么：机器处于空闲状态，正在执行 idle 循环，由 cpu_idle() 循环调用 default_idle()。此时定时器中断产生了，它引起了对定时器的处理。tulip_timer() 这个定时器处理函数被调用，而就是它引用了空指针。甚至可以通过偏移量（像 0x128/0x1c4 这些出现在函数左侧的数字）找出导致问题的语句。

寄存器上下文信息可能同样有用，尽管使用起来不那么方便。如果你有函数的汇编代码，这

些寄存器数据可以帮助你重建引发问题的现场。在寄存器中发现一个本不应该出现的数值可能会在黑暗中给你带来第一丝光明。在上面的例子中，我们可以查看是哪个寄存器包含了 NULL（一个所有位都为零的数值），进而找出是函数的哪个变量的值不正常。一般在这种情况下问题往往是竞争引起的，在本例中，是指定时器和这块网卡驱动的其他部分之间的竞争。调试一个竞争条件往往很有挑战性。

18.4.1 ksymoops

前面列举的 oops 可以说是一个经过解码的 oops，因为内存地址都已经转换成了它们对应的函数。下面是其未解码版本：

```
NIP: C013A7F0 LR: C013A7F0 SP: C0685E00 REGS: c0905d10 TRAP: 0700
Not tainted
MSR: 00089037 EE: 1 PR: 0 FP: 0 ME: 1 IR/DR: 11
TASK = c0712530[0] 'swapper' Last syscall: 120
GPR00: C013A7C0 C0295E00 C0231530 0000002F 00000001 C0380CB8 C0291B80 C02D0000
GPR08: 000012A0 00000000 00000000 C0292AA0 4020A088 00000000 00000000 00000000
GPR16: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
GPR24: 00000000 00000005 00000000 00001032 C3F7C000 00000032 FFFFFFFF C3F7C1C0
Call trace: [c013ab30] [c0020744] [c001b864] [c0007e80] [c00061c4]
[c0007b84] [c0007bf8] [c0003ae8]
```

回溯线索中的地址需要转化成有意义的符号名称才方便使用。这需要调用 ksymoops 命令，并且还必须提供编译内核时产生的 System.map。如果使用的是模块，还需要一些模块信息。ksymoops 通常会自行解析这些信息，所以一般可以这样调用它：

```
ksymoops saved_oops.txt
```

然后该程序就会吐出解码版的 oops。如果 ksymoops 无法找到默认位置上的信息，或者想提供不同信息，该程序可以接受许多参数。ksymoops 的使用手册上提供了许多说明信息，使用之前最好先行查阅。ksymoops 一般会随 Linux 发行版本提供。

18.4.2 kallsyms

谢天谢地，现在已经无须使用 ksymoops 工具了，这是一个了不起的工作。因为尽管开发者使用它的时候一般很少出现问题，但是最终用户常常会错误地匹配 System.map 文件或错误地对 oops 进行解码。

开发版的 2.5 版内核引入了 kallsyms 特性，它可以通过定义 CONFIG_KALLSYMS 配置选项启用。该选项存放着内核镜像中相应函数地址的符号名称，所以内核可以打印解码好的跟踪线索。相应地，解码 oops 也不再需要 System.map 或者 ksymoops 工具了。但是，这样做会使内核变大一些，因为从函数的地址到符号名称的映射必须永久地驻留在内核所映射的内存地址上。然而，不管是在开发的过程中还是在部署的过程中，占用这些内存都是值得的。配置选项 CONFIG_KALLSYMS_ALL 表示不仅存放函数名称，还存放所有的

符号名称。但一般只有那些特殊的调试器才会有此需要。CONFIG_KALLSYMS_EXTRA_PASS 选项会引起内核构建过程中再次忽略内核的目标代码。这个选项只有在调试 kallsyms 本身时才会有用。

18.5 内核调试配置选项

在编译的时候，为了方便调试和测试内核代码，内核提供了许多配置选项。这些选项都在内核配置编辑器的内核开发（Kernel hacking）菜单项中，它们都依赖于 CONFIG_DEBUG_KERNEL。当开发内核的时候，作为一种练习，不妨打开所有这些选项。

有些选项确实有用，应该启用 slab layer debugging（slab 层调试选项）、high-memory debugging（高端内存调试选项）、I/O mapping debugging（I/O 映射调试选项）、spin-lock debugging（自旋锁调试选项）和 stack-overflow checking（栈溢出检查选项）。其中最有用的一个是 sleep-inside-spinlock checking（自旋锁内睡眠选项），这些选项确实能完成不少调试工作。

从 2.5 版开始，为了检查各类由原子操作引发的问题，内核提供了极佳的工具。回忆一下第 9 章，原子操作指那些能够不分隔执行的东西；在执行时不能中断否则就是完不成的代码。正在使用一个自旋锁或禁止抢占的代码进行的就是原子操作。在进行此类操作的时候，代码不能睡眠——使用锁时睡眠是引发死锁的元凶。

托内核抢占的福，内核提供了一个原子操作计数器。它可以被配置成一旦在原子操作过程中进程进入睡眠或者做了一些可能引起睡眠的操作，就打印警告信息并提供追踪线索。所以，包括正使用锁的时候调用 schedule()，正使用锁的时候以阻塞方式请求分配内存和在引用单 CPU 数据时睡眠在内，各种潜在的 bug 都能够被探测到。这种调试方法捕获了大量 bug，它也受到了大家极力推荐使用。

下面这些选项可以最大限度地利用该特性：

```
CONFIG_PREEMPT=y
CONFIG_DEBUG_KERNEL=y
CONFIG_KALLSYMS=y
CONFIG_DEBUG_SPINLOCK_SLEEP=y
```

18.6 引发 bug 并打印信息

一些内核调用可以用来方便标记 bug，提供断言并输出信息。最常用的两个是 BUG() 和 BUG_ON()。当被调用时，它们会引发 oops，导致栈的回溯和错误信息的打印。这些声明会导致 oops 跟硬件的体系结构是相关的。大部分体系结构把 BUG() 和 BUG_ON() 定义成某种非法操作，这样自然会产生需要的 oops。可以把这些调用当做断言使用，想要断言某种情况不该发生：

```
if (bad_thing)
    BUG();
```

或者使用更好的形式：

```
BUG_ON(bad_thing);
```

多数内核开发者相信 `BUG_ON()` 比 `BUG()` 更清晰、更可读，而且 `BUG_ON()` 会将其声明作为一个语句放入 `unlikely()` 中。请注意，有些开发者在讨论是否能用一个编译选项将 `BUG_ON()` 声明在编译时剔除，以便能在嵌入内核中节约空间。这就意味着你可以放心地使用 `BUG_ON()`，而不用担心 `BUG_ON()` 内的声明可能带来的任何“不良反应”。`BUILD_BUG_ON()` 与 `BUG_ON()` 作用相同，仅在编译时调用。如果在编译阶段已提供的声明为真，那么编译将会因为一个错误而中止。

可以用 `panic()` 引发更严重的错误。调用 `panic()` 不但会打印错误消息，而且还会挂起整个系统。显然，只应该在最糟糕的情况下使用它：

```
if (terrible_thing)
    panic("terrible thing is %ld\n", terrible_thing);
```

有些时候，只是需要在终端上打印一下栈的回溯信息来帮助调试。这个时候，`dump_stack()` 就很有用了。它只在终端上打印寄存器上下文和函数的跟踪线索：

```
if (!debug_check) {
    printk(KERN_DEBUG "provide some information...\n");
    dump_stack();
}
```

18.7 神奇的系统请求键

神奇的系统请求键 (Magic SysRq key) 是另外一根救命稻草，该功能可以通过定义 `CONFIG_MAGIC_SYSRQ` 配置选项来启用。SysRq (系统请求) 键在大多数键盘上都是标准键。在 i386 和 PPC 上，它可以通过 ALT-PrintScreen 访问。当该功能被启用的时候，无论内核处于什么状态，都可以通过特殊的组合键跟内核进行通信。这种功能可以让你在面对一台奄奄一息的系统时能完成一些有用的工作。

除了配置选项以外，还要通过一个 `sysctl` 用来标记该特性的开或关。需要启用它时使用如下命令：

```
echo 1 > /proc/sys/kernel/sysrq
```

从终端上，你可以输入 `Sysrq-h` 获取一份可用的选项列表。SysRq-s 将“脏”缓冲区跟硬盘交换分区同步，SysRq-u 卸载所有的文件系统，SysRq-b 重启设备。在一行内发送这三个键的组合可以重新启动濒临死亡的系统，这比直接按下机器的 Reset 键要安全一些。

如果机器已经完全锁死了，它也可能不会再响应神奇系统请求键，或者无法完成给定的命令。不过如果运气稍好的话，这些选项或许可以保存数据或者进行调试。表 18-2 列举了所有支持的系统请求命令。

内核代码中的 `Documentation/sysrq.txt` 对此有更详细的说明。实际的实现在 `drivers/char/sysrq.c` 中。神奇系统请求键是调试和挽救垂危系统所必需的一种工具。由于该功能对终端上的任何用户都提供服务，所以在重要的机器上启用它需要三思而行。可是对于自己用于开发的机器，启用它确实帮助很大。

表 18-2 支持 SysRq 的命令

主 要 命 令	描 述
SysRq-b	重新启动机器
SysRq-e	向 init 以外的所有进程发送 SIGTERM 信号
SysRq-h	在控制台显示 SysRq
SysRq-i	向 init 以外的所有进程发送 SIGKILL 信号
SysRq-k	安全访问键：杀死这个控制台上的所有程序
SysRq-l	向包括 init 的所有进程发送 SIGKILL 信号
SysRq-m	把内存信息输出到控制台
SysRq-o	关闭机器
SysRq-p	把寄存器的信息输出到控制台
SysRq-r	关闭键盘原始模式
SysRq-s	把所有已安装文件系统都刷新到磁盘
SysRq-t	把任务信息输出到控制台
SysRq-u	卸载所有已加载文件系统

18.8 内核调试器的传奇

很多内核开发者一直以来都希望能拥有一个用于内核的调试器。不幸的是，Linus 不愿意在它的内核源代码树中加入一个调试器。他认为调试器会误导开发者，从而导致引入不良的修正。没有人能对他的逻辑提出异议——从真正理解代码出发，确实更能保证修正的正确性。然而，许多内核开发者们还是希望有一个官方发布的、用于内核的调试器。因为这个要求看起来不会马上被满足，所以许多补丁应运而生了，它们为标准内核附加上了内核调试的支持。虽然这都是一些不被官方认可的附加补丁，但它们确实功能完善，十分强大。在我们深入这些解决方案之前，先看看标准的 Linux 调试器 gdb 能够给我们一些什么帮助是一个不错的选择。

18.8.1 gdb

可以使用标准的 GNU 调试器对正在运行的内核进行查看。针对内核启动调试器的方法与针对进程的方法大致相同：

```
gdb vmlinux /proc/kcore
```

其中 vmlinux 文件是未经压缩的内核映像，不是压缩过的 zImage 或 bzImage，它存放在源代码树的根目录上。

/proc/kcore 作为一个参数选项，是作为 core 文件来用的，通过它能够访问到内核驻留的高端内存。只有超级用户才能读取此文件的数据。

可以使用 gdb 的所有命令来获取信息。举个例子，为了打印一个变量的值，你可以用下面的命令：

```
p global_variable
```

反汇编一个函数：

```
disassemble function
```

如果编译内核的时候使用了 `-g` 参数（在内核的 Makefile 文件的 CFLAGS 变量中加入 `-g`），gdb 还可以提供更多的信息。比如，你可以打印出结构体中存放的信息或是跟踪指针。当然，编译出的内核会大很多，所以不要把编译带调试信息的内核当做一种习惯。

接下来，就要说不幸的那一面了，gdb 还是有很多局限性的。它没有任何办法修改内核数据。它也不能单步执行内核代码，不能加断点。不能修改内核数据是个非常大的缺陷。尽管在必要时反汇编函数无疑是个非常有用的功能，但是能够修改数据的却更为有用。

18.8.2 kgdb

kgdb 是一个补丁，它可以让我们在远端主机上通过串口利用 gdb 的所有功能对内核进行调试。这需要两台计算机：第一台运行带有 kgdb 补丁的内核，第二台通过串行线（不通过 modem，直接连接两台机器的电缆）使用 gdb 对第一台进行调试。通过 kgdb、gdb 的所有功能都能使用：读取或修改变量值，设置断点，设置关注变量，单步执行等。某些版本的 gdb 甚至允许执行函数。

设置 kgdb 和连接串行线比较麻烦，但是一旦做完了，调试就变得很简单了。该补丁会在 Documentation/ 目录下安装很多说明文件，可以把它们挑出来研究一下。

不同体系结构、不同内核版本使用的 kgdb 由不同的人员维护，为了给需要调试的内核找到合适的补丁，还是在网上搜索一下比较好。

18.9 探测系统

如果对内核调试有丰富的经验的话，那么你会掌握一些诀窍来帮助你更进一步地探测系统从而找到想要的答案。内核调试很有挑战性，即使是一点小的暗示或者技巧都能给你很大的帮助。我们最好把它们联系起来。

18.9.1 用 UID 作为选择条件

如果你开发的是进程相关的部分，有些时候，你可以在提供替代物的同时不打破原有代码的可执行性。这在你重写重要系统调用的时候，或者在你希望进行调试时系统功能依旧健全的情况下非常有用。

举个例子，假设为了加入一个激动人心的新特性，你重写了 `fork()` 系统调用。除非第一次的尝试就完美无缺，否则系统调试就是一场噩梦。如果 `fork()` 系统调用不正常的话，压根就不用指望整个系统还能正常工作。当然，和任何时候一样，希望总是存在的。

一般情况下，只要保留原有的算法而把你的新算法加入到其他位置上，基本就能保证安全。可以利用把用户 id (UID) 作为选择条件来实现这种功能，通过这种选择条件，可以安排到底执行哪种算法：

```
if (current->uid != 7777) {  
    /* 老算法 ... */  
} else {  
    /* 新算法 ... */  
}
```

除了 UID 为 7777 以外，其他所有的用户都用的是老算法。可以创建一个 UID 为 7777 的用户，专门来测试新算法。对于要求很严格的进程相关部分的代码来说，这种方法使得测试变得容易了许多。

18.9.2 使用条件变量

如果代码与进程无关，或者希望有一个针对所有情况都能使用的机制来控制某个特性，可以使用条件变量。这比使用 UID 还来得简单，只需要创建一个全局变量作为一个条件选择开关。如果该变量为零，就使用一个分支上的代码。如果它不为零，就选择另外一个分支。可以通过某种接口提供对这个变量的操控，也可以直接通过调试器进行操控。

18.9.3 使用统计量

有些时候你需要掌握某个特定事件的发生规律。有些时候需要比较多个事件并从中得出规律。通过创建统计量并提供某种机制访问其统计结果，很容易就能满足这种需求。

举个例子，假设我们希望得到 foo 和 bar 的发生频率，那么在某个文件中，当然最好是在定义该事件的那个文件里，定义两个全局变量：

```
unsigned long foo_stat = 0;  
unsigned long bar_stat = 0;
```

每当事件发生的时候，就让相应的变量加 1。然后在觉得合适的地方输出它。比如，可以在 /proc 目录中创建一个文件，还可以新创建一个系统调用。最简单的办法当然还是通过调试器直接访问它们。

注意，这种实现并非是 SMP 安全的。理想的办法是通过原子操作进行实现。但是仅仅对于一个简单的每次加 1 的调试统计量，一般无须搞得这么麻烦。

18.9.4 重复频率限制

为了发现一个错误，开发者们往往在代码的某个部分加入很多错误检查语句（多数对应的都是一些打印语句）。在内核中，有些函数每秒都要被调用很多次。如果你在这样的函数中加入了 printk()，那么系统马上就会被显示调试信息这一个任务压得喘不过气来，很快就什么也不干了。

有两种相关的技巧可以用来防止此类问题的发生。第一种是重复频率限制，如果某种事件发生的非常频繁，而又需要观察它的整体进展情况，就可以让这种技巧施展身手了。为了避免调试信息发生井喷，可以每隔几秒执行一次打印（或者是其他任何你想完成的操作）。举个例子：

```
static unsigned long prev_jiffy = jiffies;          /* 频率限制 */

if (time_after(jiffies, prev_jiffy + 2*HZ)) {
    prev_jiffy = jiffies;
    printk(KERN_ERR "blah blah blah\n");
}
```

此例中，调试信息最多两秒打印一次。这可以让你的终端不至于被汹涌而至的调试信息洪流堵塞，也保证你的系统依旧能用。完全可以根据自己的需要，或低或高地调整这种重复频率。

如果只使用 `printk()`，可以用一个特殊的函数去限制 `printk()` 的调用频率：

```
if (error && printk_ratelimit())
    printk(KERN_DEBUG "error=%d\n", error);
```

如果频率限制生效，那么 `printk_ratelimit()` 返回 0；否则，返回非 0。默认情况下，此函数限制每 5 秒产生一条信息，但是在施加这一条件之前，可以让起始频率为 10 条信息。可以通过 `printk_ratelimit` 和 `printk_ratelimit_burst` `sysctl` 来调整这些参数。

另一种棘手的问题是你如何确认在特定情况下某段代码确实被执行了。与前面的例子不同，你想观察的不是一个实时通知。如果这种通知在被触发一次之后依旧不停地到来，那就比较麻烦了。下面这种技巧针对的就不再是如何限制重复频率了，它要实现的是发生次数限制。

```
static unsigned long limit = 0;

if (limit < 5) {
    limit++;
    printk(KERN_ERR "blah blah blah\n");
}
```

此例中，调试信息输出 5 次就封顶了。5 次之后，打印条件总是不能成立。

不管是上面提到的哪个示例，用到的变量都应该是静态的（`static`），并且应该限制在函数的局部范围以内，这样才能保证变量的值在经历多次函数调用后仍然能够保留下来。

这些例子的代码都不是 SMP 或抢占安全的，不过，只需要用原子操作改造一下就没问题了。不过，对于一个临时的调试检测来说，没必要搞得这么复杂。

18.10 用二分查找法找出引发罪恶的变更

知道 bug 是什么时候引入内核源代码的通常都是很有用的。如果你知道 2.6.33 版中出现了 bug，而能肯定 2.4.29 中没有，那么就能够很容易地对引发这个 bug 的代码变更进行定位。消灭 bug 变得唾手可得——要么取消这个变更，要么对其进行修正。

可是，很多时候并不知道到底是哪个内核版本引入了 bug。你知道当前版本里 bug 是确实存在的，不过，它好像就是存在于当前版本中。只需要花一点点力气，就能找出引发问题的代码变更了。元凶在手，消灭 bug 就指日可待了。

一开始，需要一个可靠的可复制的错误，最好是系统一启动就能查证的 bug。接下来，需要一个能确保没问题的内核（你应该能够找到）。举个例子，你知道几个月前的内核没有这种错误，那么就从那时使用的内核中选取一个。如果发现问题，说明那时就存在了，那就找更早的。找到

不含该 bug 的内核应该不会太难。

接下来需要一个肯定有问题的内核。为了简单起见，应该从已知最早出现该问题的内核开始。

现在，你就可以在问题内核和良好的内核之间使用二分法了。举个例子，假定确保没有问题的内核版本是 2.6.11，有问题的内核版本是 2.6.20。从二者的正中选取一个内核版本，比如说 2.6.15。检查 2.6.15 是否包含此 bug。如果 2.6.15 没有问题，那么就知错误是发生在此版本之后了。所以，再从 2.6.15 开始，在它和 2.6.20 正中选取下一个版本，比如说对 2.6.17 进行检查。如果 2.6.15 有问题，那么错误就可能发生在此版本之前了，那么就该选 2.6.13 作为下一个待查目标了。就这样重复筛选。

最终你肯定能把问题局限在两个相继发行的版本之间——一个包含错误而另外一个不包含。你就能够很容易地对引发这个 bug 的代码变更进行定位。

这种方式比依次对每个版本的内核进行核查要好得多。

18.11 使用 Git 进行二分搜索

Git 源码管理工具提供了一个有用的二分搜索机制。如果你使用 Git 来控制 Linux 源码树的副本，那么 Git 将自动运行二分搜索进程。此外，Git 会在修订版本中进行二分搜索，这样可以找到具体哪次提交的代码引发了 bug。很多 Git 相关的任务比较繁杂，但使用 Git 进行二分搜索并不那么的困难。一开始，你得告诉 Git 你要进行二分搜索：

```
$ git bisect start
```

然后再为 Git 提供一个出现问题的最早内核版本：

```
$ git bisect bad <revision>
```

如果当前的内核版本就是引发 bug 的罪魁祸首，那么就不必提供内核版本：

```
$ git bisect bad
```

然后，还得为 Git 提供一个最新的可正常运行的内核版本：

```
$ git bisect good v2.6.28
```

接下来，Git 将会利用二分搜索法在 Linux 源码树中，自动检测正常的内核版本和有 bug 的内核版本之间哪个版本有隐患。接着再编译、运行以及测试正被检测的版本。如果这个版本一切正常，可以运行下面的命令：

```
$ git bisect good
```

如果这个版本运行有异常——也就是说，如果证明这个给定的内核版本有 bug，可以运行：

```
$ git bisect bad
```

对于每一条命令，Git 将在每一个版本的基础上反复二分搜索源码树，并且返回所查的下一个内核版本。这个过程需要反复执行直到不能再进行二分搜索为止。Git 将最终打印出有问题的版本号。

这本来应该是一个漫长的过程，但是 Git 使得这一过程变得容易起来。如果你已经知道引发 bug 的源（比如，x86 机型的启动代码），你可以指定 git 仅仅在与错误相关的目录列表中去二分搜索提交的补丁。

```
$ git bisect start - arch/x86
```

18.12 当所有的努力都失败时：社区

或许你已经做完了所有你能想到的尝试。你在键盘上呕心沥血了几个小时——实际上，可能是无数日子，答案依旧没有眷顾你。此时，如果 bug 是在 Linux 内核的主流部分中，你可以在内核开发社区中寻求其他开发者的帮助。

你应该向内核邮件列表发送一份电子邮件，对 bug 进行完整而又简洁地描述，你的发现可能会对找到最终的答案起到帮助。毕竟，没人希望 bug 存在。

第 20 章将会重点推荐社区和它最重要的论坛——Linux 内核邮件列表（LKML）。

18.13 小结

本章讨论了内核的调试——调试过程其实是一种寻求实现与目标偏差的行为。我们考察了几种技术：从内核内置的调试架构到调试程序，从记录日志到用 git 二分法查找。因为调试 Linux 内核困难重重，非调试用户程序能比，因此，本章的资料对于试图在内核代码中牛刀小试的任何人都至关重要。

我们将在第 19 章涉及另外的话题：Linux 内核的可移植性。

不要止步！



第 19 章

可移植性

Linux 是一个可移植性非常好的操作系统，它广泛支持许多不同体系结构的计算机。可移植性是指代码从一种体系结构移植到另外一种不同的体系结构上的方便程度。我们都知道 Linux 是可移植的，因为它已经能够在各种不同的体系结构上运行了。但这种可移植性不是凭空得来的——需要在编写可移植代码时就为此付出努力并坚持不懈。现在，这种努力已经开始得到回报了，移植 Linux 到新的系统上就很容易（相对来说）完成。本章中我们将讨论如何编写可移植的代码——编写内核代码和驱动程序时，必须时刻牢记这个问题。

19.1 可移植操作系统

有些操作系统在设计时把可移植性作为头等大事之一，尽可能少地涉及与机器相关的代码。汇编代码用得少之又少，为了支持各种不同类别的体系结构，界面和功能在定义时都尽最大可能地具有普适性和抽象性。这么做最显著的回报就是需要支持新的体系结构时，所需完成的工作要相对容易许多。一些移植性非常高而本身又比较简单的操作系统在支持新的体系结构时，可能只需要为此体系结构编写几百行专门的代码就行了。问题在于，体系结构相关的一些特性往往无法被支持，也不能对特定的机器进行手动优化。选择这种设计，就是利用代码的性能优化能力换取代码的可移植性。Minix、NetBSD 和许多研究用的系统就是这种高度可移植操作系统的实例。

与之相反，还有一种操作系统完全不顾及可移植性，它们尽最大的可能追求代码的性能表现，尽可能多地使用汇编代码，压根就是只为在一种硬件体系结构使用。内核的特性都是围绕硬件提供的特性设计的。因此，将其移植到其他体系结构就等于再重新从头编写一个新的操作系统内核，而且即便进行移植，这种操作系统在其他体系结构上也会不适用。选择这种设计，就是用代码的可移植性换取代码的性能优化能力。这样的系统往往比移植性好的系统更难维护。当然，这种系统对性能的要求不见得比对可移植性系统更强，不过它们还是愿意牺牲可移植性，而不乐意让设计打折扣。DOS 和 Windows 95 便是这种设计方案的最好例证。

Linux 在可移植性这个方面走的是中间路线。差不多所有的接口和核心代码都是独立于硬件体系结构的 C 语言代码。但是，在对性能要求很严格的部分，内核的特性会根据不同的硬件体系进行调整。举例来说，需要快速执行的和底层的代码都与硬件相关并且是用汇编语言写成的。这种实现方式使 Linux 在保持可移植性的同时兼顾对性能的优化。当可移植性妨碍性能发挥的时候，往往性能会被优先考虑。除此之外，代码就一定要保证可移植性。

一般来说，暴露在外的内核接口往往是与硬件体系结构无关的。如果函数的任何部分需要针对特殊的体系结构（无论是出于优化的目的还是作为一种必需的选择）提供支持的时候，这些部

分都会被安置在独立的函数中，等待调用。每种被支持的体系结构都实现了一个与体系结构相关的函数，而且会链接到内核映像之中。

调度程序就是一个好例子。调度程序的主体程序存放在 `kernel/sched.c` 文件中，用 C 语言编写，与体系结构无关。可是，调度程序需要进行的一些工作，比如说切换处理器上下文和切换地址空间等，却不得不依靠相应的体系结构完成。于是，内核用 C 语言编写了函数 `context_switch()` 用于实现进程切换，而在它的内部，则会调用 `switch_to()` 和 `switch_mm()` 分别完成处理器上下文和地址空间的切换。

而对于 Linux 支持的每种体系结构，它们的 `switch_to()` 和 `switch_mm()` 实现都各不相同。所以，当 Linux 需要移植到新的体系结构上的时候，只需要重新编写和提供这样的函数就可以了。

与体系结构相关的代码都存放在 `arch/architecture/` 目录中，`architecture` 是 Linux 支持的体系结构的简称。比如说，Intel x86 体系结构对应的简称是 `x86`（这种体系结构既支持 `x86-32` 又支持 `x86-64`）。与这种体系结构相关的代码都存放在 `arch/x86` 目录下。2.6 系列内核支持的体系结构包括 `alpha`、`arm`、`avr32`、`blackfin`、`cris`、`frv`、`h8300`、`ia64`、`m32r`、`m68k`、`m68knommu`、`mips`、`mn10300`、`parisc`、`powerpc`、`s390`、`sh`、`sparc`、`um`、`x86` 和 `xtensa`。本章稍后给出的表 19-1 是一份更详尽的清单。

19.2 Linux 移植史

当 Linus 最初把 Linux 带到这个无法预测的大千世界的时候，它只能在 i386 上运行。尽管这个操作系统通用性很强，代码也写得不错，可是可移植性在那时算不上是一个关注焦点。实际上，Linus 还一度建议让 Linux 只在 i386 体系结构上驰骋。不过，人们还是在 1993 年开始把 Linux 向 Digital Alpha 体系结构上移植了。Digital Alpha 是一种高性能现代计算机体系结构，它支持 RISC 和 64 位寻址。这与 Linus 最初选的 i386 无疑是天壤之别。虽然如此，最初的这次移植工作最终还是花了将近一年时间，Alpha 机成为了 i386 后第一个被官方支持的体系结构。万事开头难，这次移植的挑战性是最大的，为了提高可移植性，内核中不少代码都被重写了^①。尽管这给整个移植带来了不小的工作量，可是效果是显著的，自此以后，移植变得简单轻松多了。

尽管第一个发行版只支持 Intel i386，但 1.2 版的内核就可以支持 Digital Alpha、Intel x86、MIPS 和 SPARC——虽然支持的不是很完善，而且带些试验性质。

在 2.0 版内核中，加入了对 Motorola 68K 和 PowerPC 的官方支持，而原 1.2 版支持的体系结构也纳入了官方支持的范畴，并且稳定下来。

2.2 版内核加入了对更多体系结构的支持，新增了对 ARMS、IBM S390 和 UltraSPARC 的支持。没过几年，2.4 版内核支持的体系结构就达到了 15 个，像 CRIS、IA_64、64 位 MIPS、HP PA_RISC、64 位 IBM S/390 和 Hitachi SH 都被加进来了。

^① 在内核开发中这很普遍。如果打算做一件事，那么就要把它做好。为了追求完美，内核开发者们是决不会介意重写大段代码的。

当前的 2.6 内核把体系结构的数目进一步提高到了 21 个，有不含 MMU 的 AVR、FR-V 和 Motorola 68k 以及 M32xxx、H8/300、IBM POWER、Xtensa，甚至还提供了用户模式 (Usermode) Linux (一个在 Linux 虚拟机上运行的内核版本)。

每一种体系结构本身就可以支持不同的芯片和机型。像被支持的 ARM 和 PowerPC 等体系结构，它们就可以支持很多不同的芯片和机型。其他的体系结构，比如说 x86 和 SPARC，它们可以支持 32 位和 64 位不同的处理器。所以说，尽管 Linux 移植到了 21 种基本体系结构上，但实际上可以运行它的机器的数目要大得多。

19.3 字长和数据类型

能够由机器一次完成处理的数据称为字。这和我们在文档中用字符 (8 位) 和页 (许多字，通常是 4KB 或 8KB) 来计量数据是相似的。字是指位的整数数目——比如说，1、2、4 或 8 等。但人们说某个机器是多少“位”的时候，他们其实说的就是该机器的字长。比如说，当人们说 Intel i7 是 64 位芯片时，他们的意思是奔腾的字长为 64 位，也就是 8 字节。

处理器通用寄存器 (general-purpose registers, GPR) 的大小和它的字长是相同的。一般来说，对于一个体系结构，它各个部件的宽度 (比如说内存总线) 最少要和它的字长一样大。虽然物理地址空间有时候会比字长小，但虚拟地址空间的大小也等于字长，至少 Linux 支持的体系结构中都是这样的^①。此外，C 语言定义的 long 类型总是对等于机器的字长，而 int 类型有时会比字长小。比如说，Alpha 是 64 位机器，所以它的寄存器、指针和 long 类型都是 64 位长度的，而 int 类型是 32 位的。Alpha 机每一次可以访问和操作一个 64 位长的数据。

字、双字以及混合

有些操作系统和处理器不把它们的标准字长称作字，相反，出于历史原因和某种主观的命名习惯，它们用字来代表一些固定长度的数据类型。比如说，一些系统根据长度把数据划分为字节 (byte, 8 位)、字 (word, 16 位)、双字 (double words, 32 位) 和四字 (quad words 64 位)，而实际上该机是 32 位的。在本书中 (在 Linux 中一般也是这样)，像我们前面所讨论的那样，一个字就代表处理器的字长。

对于支持的每一种体系结构，Linux 都要将 `<asm/types.h>` 中的 `BITS_PER_LONG` 定义为 C long 类型的长度，也就是系统的字长。表 19-1 是 Linux 支持的体系结构和它们的字长的对照表。

一般而言，Linux 对于一种体系结构都会分别实现 32 位和 64 位的不同版本。比如，在 2.6 内核的早期版本中，内核中就同时有 i386 和 x86-64，mips 和 mips64，以及 ppc 和 ppc64。但现在，经过大家的努力，这些体系结构均放在 `arch/` 目录下，每个代码库中既支持 32 位又支持 64 位。

^① 不过实际上可寻址的内存空间也可能会比字长小一些。比如，一个 64 位的体系结构虽然可能会提供 64 位的指针，但可能只会用 48 位来寻址。此外，如果支持 Intel 的 PAE，那么实际的物理内存也有比字长还大的可能。

表 19-1 Linux 支持的体系结构

体系结构	描 述	字 长
alpha	Digital Alpha	64 位
arm	ARM 和增强型 ARM	32 位
avr	AVR	32 位
blackfin	Blackfin	32 位
cris	CRIS	32 位
frv	FR-V	32 位
h8300	H8/300	32 位
ia64	IA-64	64 位
m32r	M32xxx	32 位
m68k	Motorola 68k	32 位
m68knommu	无 MMU 型 M68k	32 位
mips	MIPS	32 位和 64 位
parisc	HP PA-RISC	32 位和 64 位
powerpc	PowerPC	32 位和 64 位
s390	IBM S/390	32 位和 64 位
Sh	Hitachi SH	32 位
Sparc	SPARC	32 位和 64 位
Um	Usermode Linux	32 位和 64 位
x86	x86-32 和 x86-64	32 位和 64 位
xtensa	Xtensa	32 位

C 语言虽然规定了变量的最小长度，但是没有规定变量具体的标准长度，它们可以根据实现变化^①。C 语言的标准数据类型长度随体系结构变化这一特性不断引起争议。好的一面是标准数据类型可以充分利用不同体系结构变化的字长而无须明确定义长度。C 语言中 long 类型的长度就被确定为机器的字长。不好的一面是在编程时不能对标准的 C 数据类型进行大小的假定，没有什么能够保障 int 一定和 long 的长度是相同的^②。

情况其实还会更加复杂，因为用户空间使用的数据类型和内核空间的数据类型不一定要相互关联。sparc64 体系结构就提供了 32 位的用户空间，其中指针、int 和 long 的长度都是 32 位。而在内核空间，它的 int 长度是 32 位，指针和 long 的长度却是 64。没有什么标准来规范这些。

牢记下述准则：

- ANSI C 标准规定，一个 char 的长度一定是 1 字节。
- 尽管没有规定 int 类型的长度是 32 位，但在 Linux 当前所有支持的体系结构中，它都是 32 位的。

① 唯一的例外是 char，它的长度总是 8 位。

② 事实上对于 Linux 支持的 64 位体系结构来说，long 和 int 长度是不同的，int 是 32 位的，而 long 是 64 位的。但对于我们所熟悉的 32 位体系结构而言，两种数据类型都是 32 位的。

- short 类型也类似，在当前所有支持的体系结构中，虽然没有明文规定，但是它都是 16 位的。
- 绝不应该假定指针和 long 的长度，在 Linux 当前支持的体系结构中，它们可以在 32 位和 64 位中变化。
- 由于不同的体系结构 long 的长度不同，决不应该假设 `sizeof(int) = sizeof(long)`。
- 类似地，也不要假设指针和 int 长度相等。

操作系统常用一个简单的助记符来描述此系统中数据类型的大小。比如，64 位的 Windows 系统简称为 LLP64，它说明 long 和指针的长度都是 64 位。64 位的 Linux 系统可简记为 LP64，即 long 和指针都是 64 位。32 位的 Linux 系统简称为 ILP32，即 int、long 和指针的长度均为 32 位。这些助记符可以一目了然地显示出操作系统所提供的字长大小，因为这种方法涉及一种权衡问题。

现在依次来分析 ILP64、LP64 和 LLP64 这三种情况。ILP64 这种操作系统，int、long 和指针的大小都是 64 位。这样的数据长度使得编程变得更加容易，因为 C 语言中主要的数据类型大小是一样的（整型和指针大小的不匹配是编程中常出现的错误）。不过这样也会带来缺点，这种整型比我们平常所需的整型要大很多。在 LP64 操作系统中，程序员可以使用不同大小的整型，但必须注意整型的大小比指针类型要小。对于 LLP64 系统而言，程序员不仅要被迫接受 int 和 long 的大小相同，还要担心整型和指针之间的大小不匹配。大多数程序员都喜欢 LP64 型，即 Linux 所采用的操作系统模型。

19.3.1 不透明类型

不透明数据类型隐藏了它们的内部格式或结构。在 C 语言中，它们就像黑盒一样。支持它们的语言不是很多。作为替代，开发者们利用 typedef 声明一个类型，把它叫做不透明类型，希望其他人别去把它重新转化回对应的那个标准 C 类型。通常开发者们在定义一套特别的接口时才会用到它们。比如说用来保存进程标识符的 pid_t 类型。该类型的实际长度被隐藏起来了——尽管任何人都可以偷偷撩开它的面纱，发现它就是一个 int。如果所有代码都不显式地利用它的长度[⊖]，那么改变时就不会引起什么争议，这种改变确实可能会出现：在老版本的 Unix 系统中，pid_t 的定义是 short 类型。

另外一个不透明数据类型的例子是 atomic_t。在第 10 章中介绍过，它放置的是一个可以进行原子操作的整型值。尽管这种类型就是一个 int，但利用不透明类型可以帮助确保这些数据只在特殊的有关原子操作的函数中才会被使用。不透明类型还帮助我们隐藏了 atomic_t 类型的可用长度，但是该类型也并不总是完整的 32 位，比如在 32 位 SPARC 体系下长度就被限制。

内核还用到了其他一些不透明类型，包括 dev_t、gid_t 和 uid_t 等。

处理不透明类型时的原则是：

- 不要假设该类型的长度。这些类型在某些系统中可能是 32 位，而在其他系统中又可能是 64 位。并且，内核开发者可以任意修改这些类型的大小。
- 不要将该类型转化回其对应的 C 标准类型使用。

⊖ 显式利用长度这里指直接使用 int 类型的长度，比如说在编程时使用 `sizeof(int)` 而不是 `sizeof(pid_t)`。——译者注

- 成为一个大小不可知论者。编程时要保证在该类型实际存储空间和格式发生变化时代码不受影响。

19.3.2 指定数据类型

内核中还有一些数据虽然无须用不透明的类型表示，但它们定义成了指定的数据类型。在中断控制时用到的 flag 参数就是个例子，它应该存放在 unsigned long 类型中。

当存放和处理这些特别的数据时，一定要搞清楚它们对应的类型后再使用。把它们存放在其他（如 unsigned int 等）类型中是一种常见错误。在 32 位机上这没什么问题，可是 64 位机上就会捅娄子了。

19.3.3 长度明确的类型

作为一个程序员，你往往需要在程序中使用长度明确的数据。像操作硬件设备、进行网络通信和操作二进制文件时，通常都必须满足它们明确的内部要求。比如说，一块声卡可能用的是 32 位寄存器，一个网络包有一个 16 位字段，一个可执行文件有 8 位的 cookie。在这些情况下，数据对应的类型应该长度明确。

内核在 <asm/types.h> 中定义了这些长度明确的类型，而该文件又被包含在文件 <linux/types.h> 中。表 19-2 有完整的清单。

表 19-2 长度明确的数据类型

类 型	描 述
s8	带符号字节
u8	无符号字节
s16	带符号 16 位整数
u16	无符号 16 位整数
s32	带符号 32 位整数
u32	无符号 32 位整数
s64	带符号 64 位整数
u64	无符号 64 位整数

其中带符号的变量用得比较少。

这些长度明确的类型大部分都是通过 typedef 对标准的 C 类型进行映射得到的。在一个 64 位机上，它们看起来像：

```
typedef signed char s8;
typedef unsigned char u8;
typedef signed short s16;
typedef unsigned short u16;
typedef signed int s32;
typedef unsigned int u32;
typedef signed long s64;
typedef unsigned long u64;
```

而在 32 位机上，它们可能定义成：

```
typedef signed char s8;  
typedef unsigned char u8;  
typedef signed short s16;  
typedef unsigned short u16;  
typedef signed int s32;  
typedef unsigned int u32;  
typedef signed long long s64;  
typedef unsigned long long u64;
```

上述的这些类型只能在内核内使用，不可以在用户空间出现（比如，在头文件中的某个用户可见结构中出现）。这个限制是为了保护命名空间。不过内核对应这些不可见变量同时也定义了对应的用户可见的变量类型，这些类型与上面类型所不同的是增加了两个下划线前缀。比如，无符号 32 位整型对应的用户空间可见类型就是 `__u32`。该类型除了名字有区别外，其他方面与 `u32` 相同。在内核中你可以任意使用这两个名字，但是如果是用户可见的类型，那必须使用下划线前缀的版本名，防止污染用户空间的命名空间。

19.3.4 char 型的符号问题

C 标准表示 `char` 类型可以带符号也可以不带符号，由具体的编译器、处理器或由它们两者共同决定到底 `char` 是带符号还是不带符号。

大部分体系结构上，`char` 默认是带符号的，它可以自 -128 到 127 之间取值。也有一些例外，比如 ARM 体系结构上，`char` 就是不带符号的，它的取值范围是 0 ~ 255。

举例来说，在默认 `char` 不带符号的情况下，下面的代码实际会把 255 而不是把 -1 赋予 `i`：

```
char i = -1;
```

而另一种机器上，默认 `char` 带符号，就会确切地把 -1 赋予 `i`。如果程序员本意是把 -1 保存在 `i` 中，那么前面的代码就该修改成：

```
signed char i = -1;
```

另外，如果程序员确实希望存储 255，那么代码应该如下：

```
unsigned char = 255;
```

如果在自己的代码中使用了 `char` 类型，那么要保证在带符号和不带符号的情况下代码都没问题。如果能明确要用的是哪一个，就直接声明它。

19.4 数据对齐

对齐是跟数据块在内存中的位置相关的话题。如果一个变量的内存地址正好是它长度的整数倍，它就称作是自然对齐的。举例来说，对于一个 32 位类型的数据，如果它在内存中的地址刚好可以被 4 整除（也就最低两位为 0），那它就是自然对齐的。也就是说，一个大小为 2^n 字节的数据类型，它地址的最低有效位的后 n 位都应该为 0。

一些体系结构对对齐的要求非常严格。通常像 RISC 的系统，载入未对齐的数据会导致处理

器陷入（一种可处理的错误）。还有一些系统可以访问没有对齐的数据，只不过性能会下降。编写可移植性高的代码要避免对齐问题，保证所有的类型都能够自然对齐。

19.4.1 避免对齐引发的问题

编译器通常会通过让所有的数据自然对齐来避免引发对齐问题。实际上，内核开发者在对齐上不用花费太大心思——只有搞 gcc 的那些老兄才应该为此犯愁呢。可是，当程序员使用指针太多，对数据的访问方式超出编译器的预期时，就会引发问题了。

一个数据类型长度较小，它本来是对齐的，如果你用一个指针进行类型转换，并且转换后的类型长度较大，那么通过改指针进行数据访问时就会引发对齐问题（无论如何，某些体系结构会存在这种问题）。也就是说，下面的代码是错误的：

```
char wolf[]="Like a wolf";
char *p = &wolf[1];
unsigned long l = *(unsigned long *)p;
```

这个例子将一个指向 char 型的指针当做指向 unsigned long 型的指针来用，这会引起问题，因为此时会试图从一个并不能被 4 或 8 整除的内存地址上载入 32 位或 64 位的 unsigned long 型数据。

这种复杂的访问可能看起来有些模糊，不过通常就是如此。无论如何，这种错误出现了，所以应该小心。实际编程时错误可能不会像一些例子中那么明显或复杂。

19.4.2 非标准类型的对齐

前面提到了，对于标准数据类型来说，它的地址只要是其长度的整数倍就对齐了。而非标准的（复合的）C 数据类型按照下列原则对齐：

- 对于数组，只要按照基本数据类型进行对齐就可以了，随后的所有元素自然能够对齐。
- 对于联合体，只要它包含的长度最大的数据类型能够对齐就可以了。
- 对于结构体，只要结构体中每个元素能够正确地对齐就可以了。

结构体还要引入填补机制，这会引出下一个问题。

19.4.3 结构体填补

为了保证结构体中每一个成员都能够自然对齐，结构体要被填补。这点确保了当处理器访问结构中一个给定元素时，元素本身是对齐的。举个例子，下面是一个在 32 位机上的结构体：

```
struct animal_struct {
    char dog;                /* 1 字节 */
    unsigned long cat;       /* 4 字节 */
    unsigned short pig;     /* 2 字节 */
    char fox;                /* 1 字节 */
};
```

由于该结构不能准确地满足各个成员自然对齐，所以它在内存中可不是按照原样存放的。编

译器会在内存中创建一个类似下面给出的结构体：

```
struct animal _struct {
    char dog;                /* 1 字节 */
    u8 __pad0[3];            /* 3 字节 */
    unsigned long cat;       /* 4 字节 */
    unsigned short pig;      /* 2 字节 */
    char fox;                /* 1 字节 */
    u8 __pad1;               /* 1 字节 */
};
```

填补的变量都是为了让数据自然对齐而加入的。第一个填充物占用了 3 个字节的空間，保证 cat 可以按照 4 字节对齐。这也自动使其他小的对象都对齐了，因为它们长度都比 cat 要小。第二个（也是最后的）填充是为了填补 struct 本身的大小。额外的这个填补使结构体的长度能够被 4 整除，这样，在由该结构体构成的数组中，每个数组项也就会自然对齐了。

注意，在大部分 32 位系统上，对于任何一个这样的结构体，sizeof(animal_struct) 都会返回 12。C 编译器自动进行填补以保证自然对齐。

通常你可以通过重新排列结构体中的对象来避免填充。这样既可以得到一个较小的结构体，又能保证无须填补它也是自然对齐的。

```
struct animal _struct {
    unsigned long cat;       /* 4 字节 */
    unsigned short pig;      /* 2 字节 */
    char dog;               /* 1 字节 */
    char fox;               /* 1 字节 */
};
```

现在这个结构体只有 8 字节大小了。不过，不是任何时候都可以这样对结构体进行调整的。举个例子，如果该结构体是某个标准的一部分，或者它是现有代码的一部分，那么它的成员次序就已经被定死了，虽然内核（缺少一个正式的 ABI）相比用户空间来说，这种需求要少得多。还有些时候，因为一些原因必须使用某种固定的次序——比如说，为了提高高速缓存的命中率进行优化时设定的变量次序。注意，ANSI C 明确规定不允许编译器改变结构体内成员对象的次序[⊖]——它总是由程序员来决定的。虽然编译器可以帮助你做填充，但是，如果使用 -Wpadded flag 标志，那么将使 gcc 在发现结构体被填充时产生警告。

内核开发者需要注意结构体填补问题，特别是在整体使用时——这是指当需要通过网络发送它们或需要将它们写入文件的时候，因为不同体系结构之间所需要的填补也不尽相同。这也是为什么 C 语言没有提供一个内建的结构体比较操作符的原因之一。结构体内的填充字节中可能会包含垃圾信息，所以在结构体之间进行一字节一字节的比较就不大可能实现了。C 语言的设计者（正确的）感觉到最好还是由程序员自己为不同的情况编写比较函数，这样才能利用到结构体次序信息。

⊖ 如果让编译器随心所欲地改变结构体中各个对象的位置的话，现存的程序大部分都会崩溃。在 C 语言中，函数往往通过在结构体地址上加上偏移量来计算变量的位置。

19.5 字节顺序

字节顺序是指在一个字中各个字节的顺序。处理器在对字取值时既可能将最低有效位所在的字节当做第一个字节（最左边的字节），也可能将其当做最后一个字节（最右边的字节）。如果最高有效位所在的字节放在低字节位置上，其他字节依次放在高字节位置上，那么该字节顺序称作高位优先（big-endian）。如果最低有效位所在的字节放在高字节位置上，其他字节依次放在低字节位置上，那么就称作低位优先（little-endian）。

编写内核代码时不应该假设字节顺序是给定的哪一种（当然，如果你编写的是与体系结构相关的那部分代码就另当别论了）。Linux 内核支持的机器中使用哪一种字节顺序的都有（甚至包括一些可以在启动的时候选择字节顺序的机器），适用性强的代码应该两种字节顺序都支持。

图 19-1 是高位优先字节顺序的一个实例，图 19-2 是低位优先字节顺序的一个实例。

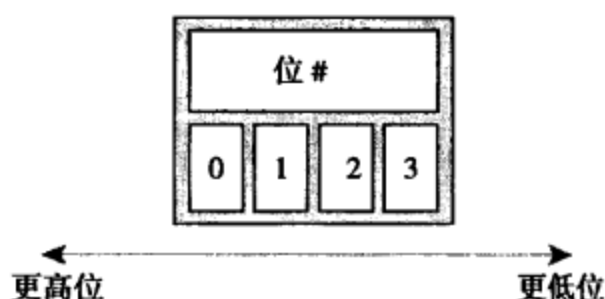


图 19-1 高位优先字节顺序

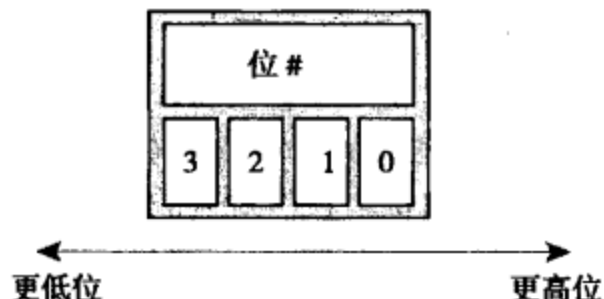


图 19-2 低位优先字节顺序

x86 体系结构，不论 32 位机还是 64 位机，使用的都是低位优先字节顺序。而其他系统大多使用高位优先字节顺序。

让我们看看在实际编程时这些概念有什么意义。让我们考察一下存放在一个 4 字节的整型中的二进制数，它的十进制对应值是 1027：

00000000 00000000 00000100 00000011

在内存中用高位优先和低位优先两种不同字节顺序存放时的比较如表 19-3 所示。

表 19-3 字节顺序比较

地 址	高 位 优 先	低 位 优 先
0	00000000	00000011
1	00000000	00000100
2	00000000	00000000
3	00000011	00000000

注意 使用高位优先的体系结构把最高字节位存放在最小的内存地址上的。这和低位优先形成了鲜明的对照。

最后一个例子，我们提供了如何判断给定的机器使用是高位优先还是低位优先字节顺序的代码：

```
int x = 1;

if (*(char *) &x == 1)
    /* 低位优先 */
```

```
else
    /* 高位优先 */
```

这段代码在用户空间和内核空间都能用。

高位优先和低位优先的历史

高位优先和低位优先源于乔纳森·斯威夫特写于 1726 年的讽刺小说《格列弗游记》。在小说中，虚构的小人国里最重要的政治问题就是应该把鸡蛋从大头敲开还是从小头敲开。那些支持从大头敲开的就是高位优先；而那些支持从小头敲开的，就是低位优先。

高位优先与低位优先的孰优孰差就好像小人国中的政治争论一样，与其说是技术问题，倒不如说是政治问题啦。

对于 Linux 支持的每一种体系结构，相应的内核都会根据机器使用的字节顺序在它的 `<asm/byteorder.h>` 中定义 `__BIG_ENDIAN` 或 `__LITTLE_ENDIAN` 中的一个。

这个头文件还从 `include/linux/byteorder/` 中包含了一组宏命令用于完成字节顺序之间的相互转换。最常用的宏命令有：

```
u32 __cpu_to_be32(u32); /* 把 cpu 字节顺序转换为高位优先字节顺序 */
u32 __cpu_to_le32(u32); /* 把 cpu 字节顺序转换为低位优先字节顺序 */
u32 __be32_to_cpu(u32); /* 把高位优先字节顺序转换为 cpu 字节顺序 */
u32 __le32_to_cpu(u32); /* 把低位优先字节顺序转换为 cpu 字节顺序 */
```

这些转换能够把一种字节顺序变为另一种字节顺序。如果两种字节顺序本来就相同（比如，希望从本地字节顺序转化为高位优先字节顺序，而处理器本身使用的就是高位优先字节顺序），那么宏就什么都不做。否则，它们就进行转换。

19.6 时间

时间测量是另一个内核概念，它随着体系结构甚至内核版本的不同而不同。绝对不要假定时钟中断发生的频率，也就是每秒产生的 jiffies 数目。相反，应该使用 HZ 来正确计量时间。这一点至关重要，因为不但不同的体系结构之间定时中断的频率不同，即使是在同一种体系机构上，两个不同版本的内核之间这种频率也不尽相同。

举个例子，在 x86 系统上，HZ 设定为 100。也就是说，定时中断每秒发生 100 次，也就是每 10ms 一次。可是在 2.6 版以前，x86 上 HZ 定为 1000。而其他体系机构上的数值各不相同：alpha 的 HZ 是 1024 而 ARM 的 HZ 是 100。

绝对不要用 jiffies 直接去和 1000 这样的数值比较，认为这样做大体上不会出问题是要不得的。计量时间的正确方法是乘以或除以 HZ。比如：

```
HZ          /* 1 秒 */
(2*HZ)      /* 2 秒 */
(HZ/2)      /* 半秒 */
(HZ/100)     /* 10ms */
(2*HZ/100)  /* 20ms */
```

HZ 定义在文件 `<asm/param.h>` 中，在前面的第 10 章中曾经讨论过。

19.7 页长度

当处理用页管理的内存时，绝对不要假设页的长度。在 x86-32 下编程的程序员往往错误地认为一页的大小就是 4KB。尽管 x86-32 机器上使用的页确实是 4KB，但是其他不同的体系结构使用的页长度可能不同。实际上有些体系结构还同时支持多种不同长度的页。表 19-4 列举了各种体系结构使用的页的长度。

当处理用页组织管理的内存时，通过 PAGE_SIZE 以字节数来表示页长度。而 PAGE_SHIFT 这个值定义了从最右端屏蔽多少位能够得到该地址对应的页的页号。举例来说，在页长为 4KB 的 x86-32 机上，PAGE_SIZE 为 4096 而 PAGE_SHIFT 为 12。它们都定义于 <asm/page.h> 中。

表 19-4 不同体系结构的页长度

体系结构	PAGE_SHIFT	PAGE_SIZE
alpha	13	8KB
arm	12, 14, 15	4KB, 16KB, 32KB
avr	12	4KB
cris	13	8KB
blackfin	12	16KB
h8300	14	4KB
	12	4KB, 8KB, 16KB, 64KB
m32r	12, 13, 14, 16	4KB
m68k	12	4KB, 8KB
m68knommu	12, 13	4KB
mips	12	4KB
mn10300	12	4KB
parisc	12	4KB
powerpc	12	4KB
s390	12	4KB
sh	12	4KB
sparc	12, 13	4KB, 8KB
um	12	4KB
x86	12	4KB
xtensa	12	4KB

19.8 处理器排序

回忆第 9 章和第 10 章，其中讨论过体系结构对指令序列的排序问题。有些处理器严格限制指令排序，代码指定的所有装载或存储指令都不能被重新排序；而另外一些体系结构对排序要求则很弱，可以自行排序指令序列。

在代码中，如果在对排序要求最弱的体系结构上，要保证指令执行顺序。那么就必须使用诸如 `rmb()` 和 `wmb()` 等恰当的内存屏障来确保处理器以正确顺序提交装载和存储指令。详情请参见第 10 章。

19.9 SMP、内核抢占、高端内存

在讨论可移植性的地方加入有关并发处理、内核抢占和高端内存的部分看起来似乎不太恰当。毕竟，这些都不是会影响到操作系统的硬件之间有所差异的那些特性；恰恰相反，它们都是 Linux 内核本身的一些功能，硬件体系结构根本感知不到它们的存在。但是，它们代表的其实都是可配置的重要选项，而你的代码应该充分考虑到对它们的支持。就是说，只有在编程时就针对 SMP/ 内核抢占 / 高端内存进行了考虑，代码才会无论内核怎样配置，都能身处安全之中。再在前面那些保证可移植性的规范下加上这几条：

- 假设你的代码会在 SMP 系统上运行，要正确地选择和使用锁。
- 假设你的代码会在支持内核抢占的情况下运行，要正确地选择和使用锁和内核抢占语句。
- 假设你的代码会运行在使用高端内存（非永久映射内存）的系统上，必要时使用 `kmap()`。

19.10 小结

要想写出可移植性好、简洁、合适的内核代码，要注意以下两点：

- 编码尽量选取最大公因子：假定任何事情都可能发生，任何潜在的约束也都存在。
- 编码尽量选取最小公约数：不要假定给定的内核特性是可用的，仅仅需要最小的体系结构功能。

编写可移植的代码需要考虑许多问题：字长、数据类型、填充、对齐、字节次序、符号、字节顺序、页大小以及处理器的加载 / 存储排序等。对于绝大多数内核开发来说，可能主要考虑的问题就是保证正确使用数据类型，虽然如此，说不定有朝一日，还是会有些与古老的体系结构有关的问题突然跳出来困扰你。所以说理解移植性的重要性，并且在开发内核过程中时刻注意编写简洁、可移植的代码是非常重要的。



第 20 章

补丁、开发和社区

Linux 的最大优势就是它有一个紧密团结了众多使用者和开发者的社区。社区能帮你检查代码，社区中的专家给你提出忠告，社区中的用户能帮你进行测试，用户还能向你反馈存在的问题。更重要的是，什么样的代码可以加入 Linux 的官方内核树也是由社区做出决定的。因此了解系统到底是怎么运作的就显得尤为重要了。

20.1 社区

如果一定要让 Linux 内核社区在现实世界中找到它的位置，那它也许会叫做内核邮件列表 (Linux Kernel Mailing List) 之家。内核邮件列表 (或者简写成 lkml) 是对内核进行发布、讨论、争辩和打口水仗的主战场。在做任何实际的动作之前，新特性会在此处被讨论，新代码的大部分也会在此处张贴。这个列表每天发布的消息超过 300 条，所以决不适合心血来潮的玩主。任何想踏踏实实研究、认认真真开发内核的人都应该订阅它 (至少要订阅它的摘要或者是它的归档资料)。单单看看这些奇才们使出的一招一式，也能让你受益匪浅了。

你可以通过向 `majordomo@vger.kernel.org` 发送下面的纯文本消息订阅这个邮件列表：

```
subscribe linux-kernel <your@email.address>
```

关于这方面更为详细的信息可以在 <http://vger.kernel.org/> 中找到，此外在 <http://www.tux.org/lkml/>，还有一个专门的 FAQ。

网上还有无数与内核相关或与普通的 Linux 使用相关的资源。<http://kernelnewbies.org/> 是一方适合内核开发初级黑客的乐土——该网站几乎能够满足所有磨刀霍霍向内核的新手的需求。还有两个网站也是不错的资源，包括 <http://www.lwn.net/>，Linux 新闻周刊，它有一个专区报道有关内核的重要新闻；<http://kernelnewbies.org/>，内核直通车，提供关于内核开发一针见血的评论。

20.2 Linux 编码风格

像所有其他大型软件项目一样，Linux 制定了一套编码风格，对代码的格式、风格和布局做出了规定。这么做不是因为 Linux 内核的风格有多么出众 (可能确实还不错) 或是你自己原来的风格有多么拙劣，而是因为保持编码风格的一致有助于提高编程效率。然而对规定编码风格还是存在一些争议，有人认为这其实无关紧要，因为无论如何，最终编译出来的目标码不会受影响。在像内核这样的大型软件项目中，涉及许许多多的开发者，编码的一致性变得至关重要。一致意味着相似和熟悉，也就意味着容易读懂，不含歧义，并且以后的代码仍旧会保持这种风格。这可以让更多的开发者读懂你的代码，也能让你读懂更多其他人编写的代码。在开源项目中，眼球自

然是越多越好。

跟选择一个唯一确定的风格相比，到底选择什么样的风格反而显得不是那么重要了。好在 Linus 早就展示出了该用什么风格，而且绝大部分代码都照这么做了。编码风格的主要规范伴随着 Linus 一贯的幽默，都记录在内核源代码树的 Documentation/CodingStyle 中了。

20.2.1 缩进

缩进风格是用制表位 (Tab) 每次缩进 8 个字符长度。这不是说用 8 个空格缩进就行了。这里的规定很明确，每次缩进通过制表位进行，每个制表位 8 个字符长度。例如：

```
static void get_new_ship(const char *name)
{
    if (!name)
        name = DEFAULT_SHIP_NAME;
    get_new_ship_with_name(name);
}
```

不知为什么，虽然违反它会对可读性带来非常大的冲击，但这个规定还是最容易被人们违反。八个字符长度的缩进能让不同的代码块看起来一目了然，特别是在连续几个小时的开发之后，效果更加明显。当然，随着缩进层数的增加，八字符制表位的左侧可用空间就所剩不多了。这是因为每行最多有 80 个字符（参见 20.2.2 节）。Linus 极其反对这样做，他认为代码不应当复杂、费解到需要两级或者三级缩进。如果真的需要多层缩进，他建议，应当重构你的代码，把复杂的层次关系（为此形成多层缩进）分解为独立的功能。

20.2.2 switch 语句

Switch 语句下属的 case 标记应该缩进到和 switch 声明对齐，这样将有助于减少 8 个字符的 tab 键带来的排版缩进，比如

```
switch (animal) {
case ANIMAL_CAT:
    handle_cats();
    break;
case ANIMAL_WOLF:
    handle_wolves();
    /* fall through */
case ANIMAL_DOG:
    handle_dogs();
    break;
default:
    printk(KERN_WARNING "Unknown animal %d!\n", animal);
}
```

当执行逻辑需要有意地从一个 case 声明尾部进入另外一个 case 声明时，对其进行评注无疑是一个普遍的（良好的）实践经验。如示例中所见。

20.2.3 空格

这一节讨论给符号和关键字加空格，而不涉及在缩进中加空格（这将在后面两节中讨论）。一般来说，Linux 的编码风格规定，空格放在关键字周围，函数名和圆括号之间无空格。例如：

```
if (foo)
while (foo)
for (i = 0; i < NR_CPUS; i++)
switch (foo)
```

相反，函数、宏以及与函数相像的关键字（例如 sizeof、Typeof 以及 alignof）在关键字和圆括号之间没有空格。

```
wake_up_process(task);
size_t nlongs = BITS_TO_LONG(nbits);
int len = sizeof(struct task_struct);
typeof(*p)
__alignof__(struct sockaddr *)
__attribute__((packed))
```

在括号内，如前所示，参数前后也不加空格。例如，下面这是禁止的：

```
int prio = task_prio( task ); /* BAD STYLE! */
```

对于大多数二元或者三元操作符，在操作符的两边加上空格。例如：

```
int sum = a + b;
int product = a * b;
int mod = a % b;
int ret = (bar) ? bar : 0;
return (ret ? 0 : size);
int nr = nr ? : 1; /* allowed shortcut, same as "nr ? nr : 1" */
if (x < y)
if (tsk->flags & PF_SUPERPRIV)
mask = POLLIN | POLLRDNORM;
```

相反，对于大多数一元操作符，在操作符和操作数之间不加空格：

```
if (!foo)
int len = foo.len;
struct work_struct *work = &dwork->work;
foo++;
-bar;
unsigned long inverted = ~mask;
```

在提领运算符的周围加上合适的空格尤为重要。正确的风格是：

```
char *strcpy(char *dest, const char *src)
```

在提领运算符的一边加上空格是不良的风格：

```
char * strcpy(char * dest, const char * src) /* BAD STYLE */
```

把提领运算符放在紧挨类型的地方也是借用 C++ 风格的一种不良作风：

```
char* strcpy(char* dest, const char* src) /* BAD STYLE */
```

20.2.4 花括号

花括号的使用不存在技术上的差异，完全是个人喜好问题，但我们还是必须宣传一致的风格。内核选定的风格是左括号紧跟在语句的最后，与语句在相同的一行。而右括号要新起一行，作为该行的第一个字符。如下例：

```
if (strcmp(buf, "NO_", 3) == 0) {  
    neg = 1;  
    cmp += 3;  
}
```

注意，如果接下来的标识符是相同语句块的一部分，那么右花括号就不单独占一行，而是与那个标识符在同一行，例如：

```
if (ret) {  
    sysctl_sched_rt_period = old_period;  
    sysctl_sched_rt_runtime = old_runtime;  
} else {  
    def_rt_bandwidth.rt_runtime = global_rt_runtime();  
    def_rt_bandwidth.rt_period = ns_to_ktime(global_rt_period());  
}
```

还有，

```
do {  
    percpu_counter_add(&ca->cpustat[idx], val);  
    ca = ca->parent;  
} while (ca);
```

函数不采用这样的书写方式，因为函数不会在内部嵌套定义：

```
unsigned long func(void)  
{  
    /* ... */  
}
```

最后，不需要一定使用括号的语句可以忽略它：

```
if (cnt > 63)  
    cnt = 63;
```

所有这些方法原理都源自 K&R[⊖]。大多数编码风格都遵循 K&R 风格，这是在那本著名的书中所使用的 C 编码风格。

[⊖] 《C 语言程序设计（第 2 版）》由 Brian Kernighan 和 Dennis Ritchie 著，这两位作者简称 K&R，该书是 C 语言的圣经，由 C 语言的发明者和他的同事合著。

20.2.5 每行代码的长度

源代码中要尽可能地保证每行代码长度不超过 80 个字符，因为这样做可使代码最适合在标准的 80×24 的终端上显示。事实上，并不存在一个广泛接受的标准——如果代码行超过 80 应该折到下一行。有些开发者也许根本不理睬代码跨行问题，而是让编辑器以可读的方式处理代码的显示；而有些开发者会手动插入断行符来分割代码行，他们也许会在新行头插入两个 tab 键以便和原先行错开。

类似的，有些开发者会在圆括号内来分行，对齐排列函数参数，比如：

```
static void get_new_parrot(const char *name,
                          unsigned long disposition,
                          unsigned long feather_quality)
```

而另一些开发者虽然也会将参数分行输入，但却不会把它们对齐排列，而是在开头简单的加入两个标准 tab。比如：

```
int find_pirate_flag_by_color(const char *color,
                              const char *name, int len)
```

因为分行没有确定的规则，所以开发者在这点上可采取自由行动。

大多数内核贡献者（包括我在内）更愿意采用前一个例子中的方式：把大于 80 个字符的行进行拆分，尽量让新产生的行与前一行对齐。

20.2.6 命名规范

名称中不允许使用骆驼拼写法（CamelCase）、Studly Caps 或者其他混合的大小写字符。局部变量如果能够清楚地表明它的用途，那么选取 idx 甚至是 i 这样的名称都是可行的。而像 theLoopIndex 这样冗长繁复的名字不在接受之列。匈牙利命名法（在变量名称中加入变量的类别）是不必要的，绝对不允许使用——要知道这里是 C，不是 Java；用的是 Unix，不是 Windows。

而全局变量和函数应该选择包含描述性内容的名称，并且使用小写字母，必要时加上下划线区分单词。给一个全局函数起名为 atty() 会使人迷惑；而像 get_active_tty() 这样就比较容易让人接受了。这里是 Linux，不是 BSD。

20.2.7 函数

根据经验，函数的代码长度不应该超过两屏，局部变量不应超过 10 个。一个函数应该功能单一并且实现精准。将一个函数分解成一些更短小的函数的组合不会带来危害。如果你担心函数调用导致的开销，可以使用 inline 关键字。

20.2.8 注释

代码的注释非常重要，但注释必须按照正确的方式进行。一般情况下，你应该描述的是你的代码要做什么和为什么要做，而不是具体通过什么方式实现的。怎么实现应该由代码本身展现。如果你不是这样做的，那么应该回过头去考虑一下你写的东西了。此外，注释不应该包含谁写了

哪个函数、修改日期和其他那些琐碎而无实际意义的内容。这些信息应该集中在文件最开头的地方。

虽然 gcc 也支持 C++ 风格的注释符号，但内核只使用 C 风格的注释符号。内核中一条注释看起来像是这样：

```
/*
 * get_ship_speed() - return the current speed of the pirate ship
 * We need this to calculate the ship coordinates. As this function can sleep,
 * do not call while holding a spinlock
 */
```

在注释中，重要信息常常以“XXX：”开头，而 bug 通常以“FIXME：”开头，就像：

```
/*
 * FIXME: We assume dog == cat which may not be true in the future
 */
```

内核包含一套自动文档生成工具。它源自 GNOME-doc，略加修改后命名为 Kernel-doc。如果想要生成独立的 HTML 格式文档，运行

```
make htmldocs
```

如果想要 postscript 格式的话，用下列命令：

```
make psdocs
```

你也可以按照特定的格式对你的函数进行注解，这样该工具也可以为你的函数服务：

```
/**
 * find_treasure find 'X marks the spot'
 * @map _treasure map
 * @time - time the treasure was hidden
 *
 * Must call while holding the pirate_ship_lock.
 */
void find_treasure(int map, struct timeval *time)
{
    /* ... */
}
```

有关此方面更多的细节请参看 Documentation/kernel-doc-nano-HOWTO.txt 文件。

20.2.9 typedef

内核开发者们强烈反对使用 typedef 语句。他们的理由是：

- typedef 掩盖了数据的真实类型。
- 由于数据类型隐藏起来了，所以很容易因此而犯错误，比如以传值的方式向栈中推入结构。
- 使用 typedef 往往是因为想要偷懒。[⊖]

⊖ 有些程序员往往是为了少敲打几次键盘而使用 typedef，比如 typedef unsigned char uchar。而这种缩写可能会引发理解和一致性上的问题，所以仅仅出于此目的而使用 typedef 被作者视为懒惰行为。——译者注

无论如何，就算是为了别惹人耻笑吧，尽量少用 typedef。

当然，typedef 也有它施展身手的时候：当需要隐藏变量与体系结构相关的实现细节的时候，当某种类型将来有可能发生变化，而现有程序必须要考虑到向前兼容问题的时候，都需要 typedef。使用 typedef 要谨慎，只有在确实需要的时候再用它；如果仅仅是为了少敲打几下键盘，别使用它。

20.2.10 多用现成的东西

请勿闭门造车。内核本身就提供了字符串操作函数、压缩函数和一个链表接口，所以请使用它们。

不要为了使现存接口更通用化而对它们进行新的封装。你经常会发现，当把一段代码从某个操作系统移植到 Linux 上的时候，表面好像看起来根本没什么问题，可是隐藏在接口下面复杂的函数调用却往往是与它原有的内核相关的。没人愿意面对这些问题，所以请直接使用内核提供的接口。

20.2.11 在源码中减少使用 ifdef

我们不赞成在源码中使用 ifdef 预处理指令。你绝不应该在自己的函数中使用如下的实现方法：

```
...
#ifdef CONFIG_FOO
    foo();
#endif
...
```

相反，应该采取的方法是在 CONFIG_FOO 没定义的时候让 foo() 函数为空。

```
#ifdef CONFIG_FOO
static int foo(void)
{
    /* ... */
}
#else
static inline int foo(void) { }
#endif /*CONFIG_FOO*/
```

这样，你在任何情况下都能调用 foo() 了。让编译器去做这些工作好了。

20.2.12 结构初始化

结构初始化的时候必须在它的成员前加上结构标识符。这种初始化能避免错误地使用其他结构而引发一个初始化错误。它也支持使用忽略值。不幸的是，C99 标准改用了一种丑陋的格式来表示这种标识符，于是 gcc 就再也不支持原来 GNU 风格的标识符了，尽管它看起来确实要更帅一些。结果，内核代码现在必须都要使用新的 C99 标识符格式了，不管它有多难看：

```
struct foo my_foo = {
    .a = INITIAL_A,
```

```
.b      = INITIAL_B,  
};
```

其中 `a` 和 `b` 是结构体 `foo` 的成员，而 `INITIAL_A` 和 `INITIAL_B` 是它们对应的初始值。如果一个字段没有给初始值，那么它就会被设置为 ANSI C 规定的默认值（如指针被设为 `NULL`，整型被设为 `0`，浮点数被设置为 `0.0`）。举例来说，如果 `foo` 结构体还有一个 `int` 型的 `c` 成员，那么上面的初始化语句执行之后 `c` 会被设置为 `0`。

20.2.13 代码的事后修正

即使你得到了一段与内核编码风格风马牛不相及的代码，也不用发愁。只消抬抬手，`indent` 工具就能帮你解决它。`indent` 是一个在大多数 Linux 系统中都能找到的好工具，它可以按照指定的方式对源代码进行格式化。默认情况下它按照不怎么好看的 GNU 编码风格格式化代码。想要用 Linux 内核编码风格，执行下列命令：

```
indent -kr -i8 -ts8 -sob -l80 -ss -bs -psl <file>
```

这样就能调用该工具按内核编码风格对你的代码进行格式化了。此外，还可以通过 `scripts/Lindent` 自动按照所需的格式调用 `indent`。

20.3 管理系统

内核黑客就是那些从事内核开发工作的人。做这些工作有些人是因为钱，有些人是因为嗜好，但几乎所有人都是为了从中找到快乐。所有做出卓越贡献的黑客都能在源代码树根目录上的 `CREDITS` 文件中留名。

内核中几乎每个部分都对应一个维护者。维护者是指一个或几个对内核特定部分负责的人。比如，每个单独的驱动程序都对应一个维护者。每个内核子系统（如网络）也有一个维护者。驱动程序和子系统的维护者也能在源代码树根目录上的 `MAINTAINERS` 文件中找到。

还有一类特殊的维护者称作内核维护者。这些人负责维护的实际上就是代码树本身。以前，由 `Linus` 自己负责维护开发版的内核（乐趣尽在此中），稳定版最开始的一段时间也由他来维护。等到该内核稳定下来了，他就会把火炬传递给最好的内核开发者中的一些人手上，由这些人负责维护该代码树，而 `Linus` 会转身启动下一开发版本的内核开发工作。在 2.6 内核继续保持稳定的“新世界秩序”前提下，`Linus` 仍然维护着 2.6 系列的内核。另外的开发者以严格的“发现 bug-修订 bug”模式维护 2.4 系列内核。

20.4 提交错误报告

如果碰到了 `bug`，最理想的应对无疑是写出修正代码，创建补丁，测试后提交它，这个流程在 20.5 节会详细介绍。当然，也可以报告这个问题，然后让其他人替你解决。

提交一个错误报告最重要的莫过于对问题进行清楚的描述。要讲清楚症状、系统输出信息、完整并经过解码的 `oops`（如果有的话）。更重要的是，你应该尽可能地提供能够准确地重现这个错误的步骤，并提供你的机器的硬件配置基本信息。

然后再来考虑把这个错误报告发送给谁。在内核源代码书的根目录中，MAINTAINERS 文件列举出了每个相关的设备驱动程序和子系统的单独信息——接收关于其所维护的代码的所有问题。如果找不到对此问题感兴趣的人，那么就把它报告给位于 `linux-kernel@vger.kernel.org` 的内核邮件列表。即使你已经找到维护者了，贴一份副本在那里也不会有什么坏处。

文档 REPORTING-BUGS 和 Documentation/oops-tracing.txt 中有更多相关信息。

20.5 补丁

对内核的任何修改都是以补丁的形式发布的，而补丁其实是 GNU diff(1) 程序的一种特定格式的输出，该格式的信息能够被 patch(1) 程序接受。

20.5.1 创建补丁

创建补丁最简单的办法是通过两份内核源代码进行，一份源码，另一份是加进了所修改部分的源代码。一般会给原来的内核代码起名 `linux-x.y.z`（其实就是把源代码包解压缩后所得到的文件夹），而修改过的就起名为 `linux`。然后利用下面的命令通过这两份代码创建补丁：

```
diff -urN linux-x.y.z/ linux/ > my-patch
```

你可以在自己的目录下运行该命令，一般都是在 `home` 目录下，而不是在 `/usr/src/linux` 目录下进行这种操作，所以不一定必须具备超级用户权限。通过 `-u` 参数指定使用特殊的 diff 输出格式。否则得到的 patch 格式怪异，一般人都无法看懂。`-r` 参数保证会遍历所有子目录进行操作，而 `-N` 参数指明做出修改的源代码中所有新加入的文件在 diff 操作时会包含在内。另外，如果想对一个单独的文件进行 diff，你也可以这么做：

```
diff -u linux-x.y.z/some/file linux/some/file > my-patch
```

注意，在你自己代码所在的目录下执行 diff 很重要。这样创建的补丁别人用起来更方便，哪怕他们的目录名字叫 `differ` 也没问题。执行一个这样生成的补丁，只需要在你自己代码树的根目录执行下列命令就可以了：

```
patch -p1 < ../my-patch
```

在这个例子中，补丁的名字叫 `my-patch`，它位于当前目录的上一级目录中。`-p1` 参数用来剥去补丁中头一个目录的名称。这么做的好处是可以在打补丁的时候忽略创建补丁的人的目录命名习惯。

`diffstat` 是一个很有用的工具，它可以列出补丁所引起的变更的统计（加入或移去的代码行）。输出关于补丁的信息，执行：

```
diffstat -p1 my-patch
```

在向 `lkml` 贴出自己的补丁时，附上这份信息往往会很有用。由于 `patch(1)` 会忽略第一个 diff 之前的所有内容，所以你甚至可以在 `patch` 的最前面直接加上简短的说明。

20.5.2 用 Git 创建补丁

如果你用 Git 管理源代码树，你照样需要用 Git 创建补丁——也就是没有必要按部就班地把上述手工的步骤操作一遍，但是你需要忍受 Git 的复杂性。用 Git 创建补丁并不是什么难事，只需要两个过程。首先，你必须是修改者，然后在本地提交你的修改。把修改提交到 Git 树与提交到标准的源代码树并没有什么两样。你根本不需要专门做任何事情去编辑存放在 Git 中的文件。你做出修改后，就需要把所做的修改提交到你的 Git 版本库：

```
git commit -a
```

-a 参数表示提交所有的修改。如果你仅仅想提交某个指定文件的修改，则如下示例：

```
git commit some/file.c
```

但是即使有了 -a 参数，Git 并不立即提交新文件，直到把它们添加到版本库中才提交。要增加一个文件，然后再提交（以及其他所有的修改），则输入如下两条命令：

```
git add some/other/file.c
git commit -a
```

当执行 Git 的 commit 命令时，Git 会要求输入一个更改日志。你应该尽量填写的详细和完整，清楚地解释修改缘由。（我们将在 20.5.3 节里详细介绍修改日志里应该包含什么）你可以针对你的版本库创建多个提交。Git 的设计可谓考虑周全，多个提交甚至可以针对同一文件，每个提交的创建各自独立。当在源码树中有一个（或两个）提交时，可以为每个提交创建一个补丁，可以像 20.5.1 节所描述的那样来处理这个补丁：

```
git format-patch origin
```

对于所有的提交，这样产生的补丁放在你的版本库中而不是原始树中。Git 产生的补丁位于源代码树的根目录中。如果只想为最后第 N 次提交产生补丁，则可以执行下列命令：

```
git format-patch -N
```

例如，下面的命令只为最后一次提交产生一个补丁：

```
git format-patch -1
```

20.5.3 提交补丁

补丁可以按照 20.5.2 节描述的方式创建。如果补丁涉及了某个特定的驱动程序或子系统，应该把它发给 MAINTAINER 中列举的相关部分的维护者。此外，还应该向 Linux 内核邮件列表 linux-kernel@vger.kernel.org 发送一份拷贝。只有在经过广泛的讨论之后，或者是补丁所做的修改很细微并且很容易就能保证正确的时候，才应该向内核维护者（比如说 Linus）提交。

一般包含一份补丁的邮件，它的主题一栏内容应该以 “[PATCH] 简要说明” 的格式写出。邮件的主体部分应该描述所做的改变的技术细节，以及要做这些的原因，越详细越准确越好。在 E-mail 中还要注明补丁对应的内核版本。

内核开发者们都希望能通过邮件阅读补丁，并且能够将其保存为一个单独文件。因此，最好

把补丁直接插入邮件，放在所有信息的最后。还要小心一些，差劲的邮件客户端工具，它们会加入信息或者改变邮件的格式；这会导致补丁出错，从而引起其他开发者的不满。如果你用的邮件客户端工具也有类似表现，就检查一下，看它是否有“Insert Inline”、“Preformat”或类似功能。如果有的话，就用纯文本方式把你的补丁贴到邮件上作为附件，不要对它做什么编码工作。

如果你的补丁很大或者包含对几个不同的逻辑的修改，那么应该将你的补丁分成几块，每块对应一个逻辑。比如你在补丁中引入了一个新的 API，并且同时对几个驱动程序进行了修改以便利用它，那么应该把该补丁一分为二（先是新的 API，然后是对驱动程序的修改），邮件也写成两份。如果任何一个部分需要其他的补丁先行，要明确地注明这一点。

提交之后，保持耐心，等待答复。别因为某些反对言论而灰心——至少你还是得到回应了嘛！和其他人讨论这个问题并且在需要的时候应该提交修正过的新补丁。如果你压根就没听到回声，想想是什么出了问题，然后着手解决它。多请邮件列表和维护者们提出宝贵意见。运气好的话，未来版本的内核发行时，你可能就会看到自己做出的修改了——那可就真的该恭喜你了！

20.6 小结

对于黑客而言，最可贵的品质便是渴望——就如身上痒痒，不抓不快一般的渴望和决心。本书讲述了 Linux 内核的主要部分，讨论了接口、数据结构、算法和原理。它从实践出发，以内在的视角洞悉内核，既可以满足你的好奇心，也可以帮助你开始学习内核。

不过，正如我前面所说，你上路的唯一方法是去自己读、写代码。Linux 社区不但创造了这样的条件，而且很欢迎大家这么做。好了，不管你追求什么，现在就开始去做吧。



参 考 资 料

在这里列出的书籍是对本书内容的补充。注意，本书最好的“课外阅读”参考资料就是内核源代码。作为 Linux 的使用者，我们被赋予无限制地获得全部现代操作系统源代码的权利。别把这当做是理所当然的，不要暴殄天物，要潜心钻研它。

操作系统设计书籍

这些书覆盖了操作系统设计领域，常作为大学教科书。它们包含设计一个功能健全的操作系统所需要的概念、算法、问题和解决方法。我推荐列出的所有书籍，如果非得我选择一本，那么 Deitel 的书既易理解又有趣可读，它是我的首选。

H. Deitel、P. Deitel 以及 D. Choffnes 的《Operating Systems》，Prentice Hall，2003。该书对操作系统的原理进行了透彻讲述，并且列出了很多出色的实例研究，适合把理论付之于实践。

Tanenbaum, Andrew 的《Modern Operating Systems》，Prentice Hall，2007。该书深刻揭示了标准操作系统设计精髓，其中也不乏讨论许多今天流行的现代操作系统，如 Unix 和 Windows。

Tanenbaum, Andrew 的《Operating Systems : Design and Implementation》，Prentice Hall，2006。该书精辟地介绍了如何设计和实现一个类 Unix 操作系统——Minix。

Silberschatz A.、P. Galvin 和 G. Gagne 的《Operating System Concepts》，John Wiley and Sons，2008。由于该书封面图片看起来和恐龙有关，所以也被称为“恐龙宝典”。这是一本介绍操作系统设计的好书，而且频繁再版，版版经典。

Unix 内核书籍

这些书主要针对 Unix 内核设计与实现，前五本讨论 Unix 特定版本，后两本关注所有 Unix 版本的通用特点。如果你只买其中的两本，那么我坚持推荐最后两本：

Bach, Maurice 的《The Design of the Unix Operating System》，Prentice Hall，1986。这是一本讨论 Unix 系统 V2 版本的优秀书籍。

McKusick M.、K. Bostic、M. Karels 和 J. Quarterman 的《The Design and Implementation of 4.4BSD Operating System》，Addison-Wesley, 1996。操作系统设计者自己编写的讨论设计 4.4BSD 系统的优秀书籍。

McKusick, M. 和 G. Neville-Neil 的《The Design and Implementation of the FreeBSD Operating System》，Addison-Wesley, 2004。这是一本讨论 FreeBSD 5.2 系统设计与实现的优秀书籍。

McDougall, R 和 J. Mauro 的《Solaris Internals: Solaris and OpenSolaris Kernel Architecture》，Prentice Hall，2006。这是一本对 Solaris 内核的核心子系统和算法进行精彩讨论的优秀书籍。

Cooper C. 和 C. Moore 的《HP-UX 11i Internals》，Prentice Hall, 2004。这本书对 HP-UX 和 PA-RISC 体系结构的内幕进行了初探。

Vahalia, Uresh 的《Unix Internals: The New Frontiers》，Prentice Hall, 1995。这是一本关于现代 Unix 系统特色的优秀书籍，主要介绍了线程管理和内核抢占等功能。

Schimmel, Curt 的《UNIX Systems for Modern Architectures: Symmetric Multiprocessing and Caching for Kernel Programmers》，Addison-Wesley, 1994。该书揭示了在现代体系结构上实现现代 Unix 操作系统可能要面对的种种危险。强烈推荐阅读这本图书。

Linux 内核书籍

以下书籍与本书一样讨论 Linux 内核。有点可惜的是，这方面的好书不多。不过，我推荐阅读下列两本。

Benvenuti, Christian 的《Understanding Linux Network Internals》，O'Reilly and Associates, 2005。这本书对 Linux 网络进行了深入分析。

Corbet, J. 、A. Rubini 和 G. Kroah-Hartman 的《Linux Device Drivers》，O'Reilly and Associates, 2005。这本书讨论了在 2.6 内核上如何编写驱动程序，尤其是针对各种设备重点描述了所支持的编程接口，这是一本经典的介绍编写驱动程序的图书。

关于其他系统的内核书

了解你的敌人——错误和竞争对手——才能处于不败之地。下列书籍讨论了非 Linux 系统的设计与实现。你可以从中获得经验或教训。

Kogan M. 和 H. Deitel 的《The Design of OS/2》，Addison-Wesley, 1996。该书介绍了 OS/2 2.0 的设计。

Singh, Amit 的《Mac OS X Internals : A Systems Approach》，Addison-Wesley Professional, 2006。该书对整个 Mac OS X 系统进行了完整论述，既有深度又有广度。

Solomon, D. 和 M. Russinovich 的《Windows Internals : Covering Windows Server 2008 and Windows Vista》，Microsoft Press, 2009。这是一本介绍非 Unix 操作系统的有趣书籍。

关于 Unix API 的书籍

深入探索 Unix 系统的 API 函数不仅对编写优秀的用户空间应用程序很关键，同时也对理解内核功能大有帮助。

Love, Robert 的《Linux System Programming》，O'Reilly and Associates, 2007。该书作者负责 Linux 的系统级编程，该书涵盖了 Linux 系统调用以及 libc API 等内容，并对 Linux 所具有的一些技巧和方式给予关注。

Stevens W. R. 和 S. Rago 的《Advanced Programming in the UNIX Environment》，Addison-Wesley, 2008。该书是讨论 Unix 系统调用接口的权威书籍。

Stevens W. Richard 的《UNIX Network Programming Volume 1》，Prentice Hall, 2004。这是一本关于 Unix 系统套接字 API 的经典教材。

关于 C 语言编程的书

Linux 内核以及 Linux 系统的其他很多软件都是用 C 语言编写的。下列两本书值得推荐阅读。

Kernighan B. 和 D.Richie 的《The C Programming Language》，Prentice Hall, 1988。这是一本介绍 C 语言编程的权威书籍，由 C 的发明者及其合作者编写。

van der Linden, Peter 的《Expert C Programming》，Prentice Hall, 1994。该书对不易理解的 C 细节给予了难能可贵的讨论，作者的语言风趣幽默。

其他书籍

下面的书籍严格说可能和操作系统无关，但是这些书无疑从某些层面影响了操作系统。

Hofstadter, Douglas 的《G·del, Escher, Bach : An Eternal Golden Braid 》，Basic Books, 1999。这是一本对人类思想进行深刻研究的必备书籍，它的内容覆盖众多主题，其中也包含计算机科学。

Knuth, Donald 的《The Art of Computer Programming, Volume 1》，Adderson-Wesley, 1997。这是一本无价的介绍计算机科学算法的基础书籍，其中介绍了内存管理中的最佳适应算法和最坏适应算法。

Web 站点

Kernel.org。内核代码库的官方站点，同时也容纳了核心内核黑客的大量补丁包。网址是：<http://www.kernel.org>。

Linux Weekly News。出色的新闻站点，对一周 Linux（包括内核）所发生的新闻给予睿智、准确的评论。强烈推荐浏览。网址是：www.lwn.net。

OS 新闻。包括关于操作系统的新闻，也包括原创文章、访谈以及 reviews。www.osnews.com。



本书详细描述了Linux内核的设计与实现。内核代码的编写者、开发者以及程序开发人员都可以通过阅读本书受益，他们可以更好地理解操作系统原理，并将其应用在自己的编码中以提高效率和生产率。

本书详细描述了Linux内核的主要子系统和特点，包括Linux内核的设计、实现和接口，从理论到实践涵盖了Linux内核的方方面面，可以满足读者的各种兴趣和需求。

作者是一位Linux内核核心开发人员，他分享了在开发Linux 2.6内核过程中颇具价值的知识和经验。本书的主题包括进程管理、进程调度、时间管理和定时器、系统调用接口、内存寻址、内存管理和页缓存、VFS、内核同步、移植性相关的问题以及调试技术。同时本书也涵盖了Linux 2.6内核中颇具特色的内容，包括CFS调度程序、抢占式内核、块I/O层以及I/O调度程序。

本书新增内容包括：

- 增加一章专门描述内核数据结构（第6章）；
- 详细描述中断处理程序和下半部机制；
- 扩充虚拟内存和内存分配的内容；
- 调试Linux内核的技巧；
- 内核同步和锁机制的深度描述；
- 提交内核补丁以及参与Linux内核社区的建设性建议。

Linux 内核设计与实现

Linux Kernel Development
Third Edition

原书第3版

客服热线：(010) 88378991, 88361066
购书热线：(010) 68326294, 88379649, 68995259
投稿热线：(010) 88379604
读者信箱：hzsj@hzbook.com

PEARSON

www.pearsonhighered.com



上架指导：计算机/程序设计

ISBN 978-7-111-33829-1



9 787111 338291

华章网站 <http://www.hzbook.com>

网上购书：www.china-pub.com

封面设计 范华明

定价：69.00元